Royal Holloway, University of London

Department of Computer Science

MSc Individual Project

Satisfiability Modulo Theories and Bounded Model Checking for AI Planning

A dissertation submitted in partial fulfillment of the requirements for the degree of Master of Science in Artificial Intelligence.

September 9, 2025

Declaration

This dissertation has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 18473 (whole document)

Page Count: 49 (47 +2 pages of bibliography)

Date of Submission: September 9, 2025

Acknowledgements

I would like to sincerely thank my supervisor. When I first came to the topic, it looked quite daunting for me but his encouragement and belief in my abilities gave me the confidence to begin. Throughout the project, his trust in my progress was a constant source of motivation. Whenever I felt stuck, he offered the crucial insights and ideas that helped me move forward. His suggestion to include a comparative analysis with PDDL planners, in particular, was instrumental in strengthening the evaluation and adding significant depth to this dissertation.

Abstract

A dissertation investigates how satisfiability modulo theories and bounded model checking can be used to solve difficult AI planning problems. Satisfiability-based planning (also known as SAT based planning) has been a powerful approach. However, it is largely inefficient when it comes to planning problems from real-world systems. To deal with this limitation, this project presents a methodology that re-frames the planning task as formal verification task. Using BMC to encode a 2D grid-world planning problem into a single logical formula, which the Z3 SMT solver then solves, will be the main implementation. A model that satisfies this formula can be construed to represent a verified plan. In addition, the planner developed in this study was modified to deal with cost constraints, and a visualisation tool was developed to demonstrate the findings better. An extensive experimental evaluation was also designed to study the performance and scalability of this SMT-based approach. The analysis systematically assesses the solver's performance as a function of grid size, plan length, goal complexity and obstacle density. To better understand these results, an added comparative benchmarks was also performed against classical heuristic search planners. The results indicate that SMT based methods are flexible but specialized planners are computationally more efficient.

Contents

Chapter 1

Introduction

1.1 Motivation

1. Introducing the Problem

The advancements in artificial intelligence have created new opportunities which are quite helpful, and at some point inevitable, in the everyday life of people. From generative AI-driven game characters to large language models, such as those from OpenAI and Google, all of them have found and hold their places by the time this thesis was written. Among them, what captured my interest most is AI's role in autonomous systems. A prime example application would be self-driving cars/taxis, pioneered and developed by companies such as Waymo and Tesla, which aim to transform traffic efficiency and safety. These systems are designed to operate in complex, stochastic environments and make decisions with significant real-world impact. The main challenge for modern autonomous systems is **AI Planning** – process of generating a list of actions for an agent to achieve a desired goal. In the real world, unfortunately, this task requires more than a simple path-finding algorithm. Considering the different unpredictable factors of the world, in this traffic – such as road accidents, car crashes, and time-dependent traffic rates – requires managing numerous interacting constraints simultaneously which includes optimizing routes for profitability and dynamically handling obstacles. The critical nature of these tasks demands solutions that are not only efficient but, most importantly, reliable and verifiably safe as in the end, human life is at stake.

2. Explain the Gap or Limitation

Historically, **Boolean Satisfiability (SAT)** problem has served as foundational approach to tackle computational problems with complex constraints. Stemming from Stephen Cook's seminal work, SAT is a classic NP-complete problem, meaning a wide variety of difficult combinatorial challenges can be transformed into its format for solving [Coo71]. Thanks to the achievements accomplished in SAT solving technology since 1990s, it has become a quite powerful engine to solve problems in hardware verification, logistics, and even aspects of AI Planning. Despite being such a powerful engine, the power of SAT is also its main limitation: it operates purely on Boolean values (true/false). Theoretically, it is true that every problem can be encoded in SAT but this becomes impractical for systems that rely on non-Boolean logic, such as integers, strings, real numbers (eg for coordinates and time), or arrays. For example, `car_position + car_speed`

`<= intersection_limit` requires complex and inefficient method, known as "bit-blasting". This process can cause an exponential growth in variables and clauses, rendering pure SAT an unwieldy and unscalable solution for the expressive, high-level constraints found in real-world AI planning [DB11]

3. Proposing the Solution/Methodology

This dissertation is motivated by the need of a bridge for the gap between the formal challenge of satisfiability-based methods and the expressive requirements of complex AI planning problems. To achieve this, and in line with the project's central aim of learning and applying formal methods, my chosen solution integrates two powerful techniques: **Satisfiability Modulo Theories (SMT)** & **Bounded Model Checking (BMC)**. Satisfiability Modulo Theories (SMT) is developed in order to overcome the limitations of traditional SAT – the limitation of boolean logic. As detailed in the background reading section, SMT extends SAT by incorporating specialized "theory solvers" that can natively reason about integers, real numbers, arrays, and other non-Boolean data types [BT18]. This provides the capability of creating much more natural and efficient representation of complex constraints possible. Considering a taxi scenario, for instance, SMT allows for a more natural and efficient representation of complex constraints such as traffic laws and optimal route calculations. [DB11]. This project specifically uses a powerful Z3 SMT solver developed by Microsoft. **Bounded Model Checking (BMC)** is a formal verification technique used to analyze a system's behavior within a finite number of execution steps, denoted by the bound k [Cla+01]. Instead of trying to prove a property holds for all time (which can be undecidable), BMC explicitly models the system's sequence of transitions up to step k and translates this behavior into a single logical formula. A satisfiability solver is then used to search this formula for a counterexample—a concrete path of length k that violates a given safety property (a rule stating something "bad" must never happen). This process is not only for finding bugs; it can also be cleverly repurposed for planning. By framing the desired goal as a property to be reached, the counterexample found by the solver actually serves as a witness: a formally verified, step-by-step plan that achieves the objective [BT18]. The central methodology of this project is to encode the AI planning problem as a Bounded Model Checking problem, which is then solved using SMT. This reframes the planning question—"Can the robot find all golds while avoiding obstacles?"—into a formal verification query: "Does a valid sequence of k moves exist that leads the robot to collect all gold pieces without entering any obstacle cells?"

4. Relevance and Impact

Through my implementation and research, I found that using SMT with BMC is a convincing approach for this AI planning problem for three main reasons. First, it provides a **strong guarantee of safety**–the resulting plan is formally proven to be correct up to the defined step limit (the bound, k). This is a practical method to guarantee reliability in real-world systems since checking every possible future action, which is impractical and sometimes even impossible in real world problems, is undecidable. Secondly, SMT offers significant flexibility in problem description as the system is not limited only to pure Boolean logic, as it can also accept integers for costs, or coordinates for the robot's position directly. As a result the model is much easier to build and, importantly, easier to modify when new rules or constraints are needed. Ultimately, this approach **alters the**

role of the developer–rather than creating a custom search algorithm, such as A^*, the focus becomes accurately specifying the rules and goals of the problem. The Z3 SMT solver, which embodies many years of optimization research, then handles the computationally difficult job of finding a valid plan [DB11].

This dissertation contributes to the field by providing a practical implementation and a comprehensive experimental evaluation of this SMT-based planning methodology. The performance and scalability of the SMT solver as a planning engine is evaluated by applying this technique on a dynamic Robot in a minefield problem. The experimental results assess how factors such as grid size, obstacle density, and plan length affect the solver's ability to find a solution. The results show how effective integrating SMT and BMC is for this class of problems, offering valuable insights into the strengths and limitations of using general-purpose constraint solvers for developing safe and reliable autonomous systems.

Additionally, to strengthen the evaluation of the proposed SMT/BMC methodology, this work includes a comparative performance analysis as suggested by my supervisor. This involves benchmarking the SMT planner against high-performance classical planners using the standard Planning Domain Definition Language (PDDL). By comparing the two different paradigms on the same problem, this analysis aims to provide a clear understanding of the practical strengths and limitations of using SMT-based techniques for this class of planning problems.

Chapter 2

Background Research

2.1 AI Planning

AI planning, also known as automated planning, is a long-standing, rich, and evolving field within Artificial Intelligence that creates capability for agents to autonomously determine a set of actions to achieve desired goal. The ultimate goal is to develop systems that can design sets of actions or "plans", which are expected to achieve specified objective.

2.1.1 Historical Context and Evolution

AI Planning emerged around 1966 from the necessity of giving autonomy to a wheeled robot, Shakey, which utilized the STRIPS (Stanford Research Institute Problem Solver) planner developed in 1971 [AG24]. The STRIPS representation was a significant contribution, originally based on first-order logic, which described actions in terms of preconditions and post-conditions (add and delete lists) [AG24]. Since then, many techniques have been introduced over 30 years, including Means-Ends Analysis, introduced by Newell and Simon in 1963; the first non-linear planner, NOAH (Nets of Action Hierarchies), by Sacerdoti in 1975; and its successor, Nonlin, in 1977 [HTD90]. Although planning research produced many systems, for years there was not any formal way to compare the performance of different planners.This changed in 1998 when the AIPS conference introduced the first international planning competition, built around shared benchmark domains and the Planning Domain Definition Language (PDDL). In 1997, a ruling committee was established to decide how the contest should proceed, which ended up choosing the Planning Domain Definition Language (PDDL) notation for planning domains. As a result, five different planners were able to compete in 1998 [McD00]. PDDL has since been extended and has become the de facto standard for specifying planning problems [AG24]. Around the same time, Kautz and Selman (1992) introduced the idea of encoding planning problems as satisfiability instances [Nar+05].In this approach, a planning problem is translated into a propositional formula, where a satisfying assignment corresponds to a valid plan. This innovation connected the field of AI planning with the long history of research into satisfiability (SAT) and, later, Satisfiability Modulo Theories (SMT) [FM09]. As SAT-based planners often outperform traditional search-based planners on certain problem types, this connection has resulted in significant performance improvements, a [Wel99].

2.1.2 Formal Models and Core Concepts in Classical Planning

To make a planning issue solvable by a computer, it must be formally defined. The starting point for this is the **state-transition system**, which has the necessary mathematical structure to describe the environment and the agent's interactions to the planner. [BG01]. A state-transition system is typically defined by a tuple (S, A, γ), where:

- **S** is a finite or discrete set of all possible **states** of the world. A state is a complete snapshot of the environment at a single point in time.
- **A** is a set of **actions** available to an agent.
- γ is a **transition function**, $S \times A \to S$, which determines the resulting state s' after an action a is executed in a state s.

While this model is general, classical AI planning relies on more structured and practical representation known as a **factored representation**. Instead of treating states as monolithic entities, a state is described by a set of **ground atoms** or state variables. For example, in the robot minefield domain, the state of the world is not just "State A," but is defined by a collection of facts like `RobotAt(2,3)`, `GoldAt(1,4)`, and `ObstacleAt(2,4)`. This factored representation is critical because it allows for a compact and logical description of the world.

Within this framework, actions are defined using a structured schema, most famously codified by the **STRIPS (Stanford Research Institute Problem Solver)** formalism [AG24]. An action schema consists of:

- **Preconditions:** A set of atoms that must be true in a state for the action to be applicable. For instance, the action `Move(from_r, from_c, to_r, to_c)` would have preconditions that the robot is at `(from_r, from_c)`, the locations are adjacent, and the destination `(to_r, to_c)` is not an obstacle.
- **Effects:** A description of how the state changes after the action is executed. This is typically split into an **add-list** (atoms that become true) and a **delete-list** (atoms that become false). For the `Move` action, the effect would be to add `RobotAt(to_r, to_c)` and delete `RobotAt(from_r, from_c)`.

This structured representation leads to a formal definition of a **planning problem** as a tuple $P = (\Sigma, s_0, g)$, where:

- Σ is the **planning domain**, which defines all possible states and action schemas.
- s_0 is the **initial state**, a fully specified set of atoms describing the starting conditions.
- g is the **goal specification**, a partially specified state describing the desired outcome (e.g., `At(Passenger, Destination)`). Any state that contains all the atoms in g is considered a goal state.

The objective of a planner is to find a **plan**, which is a sequence of actions $\pi = \langle a_1, a_2, \ldots, a_n \rangle$ that, when executed from the initial state s_0, transforms the system into a final state s_n that satisfies the goal g. The discovery of such a plan is the central challenge that connects AI planning to satisfiability problems, as the search for a valid sequence of actions can be framed as a search for a satisfying assignment to a complex logical formula [Nar+05].

2.1.3 Declarative Planning: The PDDL Standard

One significant development in the AI Planning community was the emergence of a standardized input language, which has been crucial for comparing planner performance and sharing benchmark problems. Driven by a standardization effort from the International Planning Competitions (IPCs), the **Planning Domain Definition Language (PDDL)** was established. Since then, PDDL has become the essential input language for most classical planning systems [RH09; McC03].

The core philosophy of PDDL is to provide a declarative way to specify which actions are allowed in a domain and what effects they produce, without embedding planner-specific strategies [McD+98]. This separation of the problem model from the solving algorithm is fundamental to its role as a universal communication language [McC03]. To achieve this, a PDDL planning instance is specified using two distinct files: a **domain file** and a **problem file**.

The Domain File

The domain file defines the general rules and physics of a planning environment. It specifies the types of objects that can exist, the relationships (predicates) between them, and, most importantly, the actions that can change the state of the world. A typical domain definition includes:

- `:requirements`: A set of flags indicating which features of the PDDL language are used, allowing a planner to quickly determine if it can handle the domain's level of expressiveness . Common requirements include `:strips` for basic operators, `:typing` for object types, and `:conditional-effects` for effects that only occur if certain conditions are met [McD+98].
- `:types`: Defines a hierarchy of object types. For example, `location` and `physob` (physical object) could be defined types .
- `:predicates`: Declares the predicates used to describe the state of the world, including their parameters and types. For example, `(at ?x - physob ?l - location)` could declare that a physical object `?x` can be at a location `?l` .
- `:action`: Defines an action schema, including its parameters, **preconditions** (what must be true for the action to be executable), and **effects** (the changes to the state that result from the action) . Unlike the STRIPS formalism, PDDL does not use separate add and delete lists; instead, negative effects are asserted directly, for example `(not (at B ?m))` [McD+98; THN05].

The Problem File

While the domain file describes the general "laws" of the world, the problem file specifies a concrete instance to be solved within that domain [McD+98]. It defines:

- `:domain`: The name of the associated domain file.
- `:objects`: A list of the specific, named objects that exist in this particular problem instance .

- `:init`: The initial state of the world, specified as a list of ground atomic formulae that are true at the beginning. Any predicate not listed is assumed to be false, following the closed-world assumption.
- `:goal`: A logical formula describing the desired goal state. A plan is considered a solution if it reaches a state where this goal formula is satisfied.

For example, the PDDL 1.2 manual describes a "briefcase-world" problem where the goal is to move objects between home and an office . The domain file would define the `move-briefcase` action, while the problem file would define the specific objects (`dictionary`, `paycheck`), their initial locations (`home`), and the goal of having the dictionary at the `office` while leaving the paycheck at `home` [McD+98].

Language Evolution

Subsequent versions of PDDL added numeric and temporal constructs (PDDL 2.1) and features like derived predicates and preferences (PDDL 2.2, PDDL 3) to improve expressiveness [EH04; GL05; THN05]. This ongoing development demonstrates the language's role in driving and reflecting progress in the field of automated planning.

2.1.4 Solving Paradigms: Planning as Heuristic Search

Alongside satisfiability-based methods, one of the most successful and popular approaches to classical planning is to frame the problem as a **heuristic search** [BG01; KBF02; SJJ21]. This paradigm transforms a declarative planning problem into a search for a path in a state-space graph, guided by a heuristic that is automatically extracted from the problem's definition [BG01]. In STRIPS-based planning, a state is represented by the set of atoms that hold, the initial state is given by the :init block and a goal state is any state that contains the goal atoms. An action is applicable if its preconditions are satisfied; applying it yields a successor state based on its effects [BG01]. Solving the planning problem is therefore equivalent to finding a path of actions through this state space from the initial state to any goal state.

Domain-Independent Heuristics

To efficiently navigate this large state space, the search is guided by a heuristic function, $h(s)$, which estimates the cost to reach a goal from the current state s. A strong and effective method of automatically deriving such heuristics is to solve a **"relaxed" or simplified problem** – perhaps the most common method is to ignore all negative effects (delete lists) of actions. Although finding the true optimal cost in this relaxed problem remains NP-hard, the optimal cost can be efficiently estimated and this estimate is the heuristic value[BG01]. It is possible to calculate different heuristics based on this relaxed problem. For example, **max heuristic** ($h_{\max}$), takes the maximum cost among the individual goal atoms and is **admissible**, meaning it never overestimates true cost. In contrast, the **additive heuristic** ($h_{\mathbf{add}}$) sums the costs of the individual atoms; this assumes subgoals are independent and is therefore not **not admissible** , but is often a more informative guide in practice.[BG01].

Heuristic Search Algorithms

Search algorithms like **A*** or its weighted variant, **WA*** uses these heuristics, eg hmax/hadd, as guide to find solution. This type of algorithms prioritize states using an evaluation function $f(n) = g(n) + W \cdot h(n)$, which balances the known cost to reach a state, $g(n)$, with the estimated cost to the goal,$h(n)$ [BG01]. The search behavior is determined by the properties of the heuristic function. An **admissible heuristic** is one that never overestimates the cost of reaching the goal. The A* algorithm can always find a plan of optimal (cheapest) cost if the heuristic it uses is admissible. This complete paradigm is embodied by highly influential planners like HSP and FF, which have demonstrated the effectiveness of planning as heuristic search [KBF02].

2.1.5 Used Heuristics and Planners

In this dissertation work, the heuristic search planners used for comparison are built upon the **Fast Downward** planning system and are guided by different types of domain-independent heuristics. This evaluation will focus on heuristics derived from relaxed problems and those based on landmarks.

Heuristics for Comparison

The comparison will utilize two prominent heuristic functions:

- **The Max Heuristic ($h_{\max}$):** An admissible heuristic derived from a relaxed planning graph, where the cost of a set of subgoals is taken as the maximum cost of achieving any single subgoal in the set [BG01].
- **Landmark-based Heuristics ($LM - cut$):** Landmarks are propositions that must be made true at some point in every valid solution plan. Heuristics like $LM - cut$ leverage these landmarks to provide a highly informative and admissible estimate of the distance to the goal. LAMA, one of the planners in this study, uses a landmark-based heuristic in conjunction with the well-known FF heuristic[RWH11].

Planners for Comparison

The experimental benchmarks will be run using configurations of the Fast Downward planning system. In the seminal paper on the system, **Helmert (2006)** establishes its high performance, concluding that it is "clearly competitive with the state of the art" after winning the classical track of the 2004 International Planning Competition (IPC4) [Hel06]. Two main configurations of this system will be benchmarked:

- **Optimal A* Search:** The system will be configured to perform an optimal A* search, guided by the admissible heuristics $h_{\max}$ and $LM - cut$ respectively.
- **LAMA (Lookahead-based Heuristic Search):** As **Richter et al. (2011) describe**, LAMA is a highly successful configuration of Fast Downward which won the sequential satisficing track of the IPC in 2008. It is designed not for optimal planning, but for finding high-quality solutions quickly. Its strategy is to first run a fast, **greedy best-first search** to find an initial solution, and then run a series of **weighted A* searches** to progressively improve solution quality, making it an excellent benchmark for satisficing planning performance [RWH11].

2.2 Propositional Satisfiability (SAT)

2.2.1 Core Knowledge

Propositional satisfiability, widely known as SAT, is a fundamental problem in a subgroup of Artificial Intelligence that determines whether a given propositional logic formula in conjunctive normal form (CNF formula) has a satisfying interpretation or not, by assigning true or false values to its variables. If such an assignment exists, it is called a model of the formula. The problem of determining whether a propositional formula has a model is NP-complete, a class of problems for which a proposed solution can be checked for correctness in polynomial time [Coo71].

Language and Syntax

The core language of SAT is built upon a few key syntactic objects:

- **Signature:** A set of ground atoms.
- **Literal:** An atom or a negated atom. A ground literal is one that contains no variables.
- **Clause:** A non-empty disjunction of literals (e.g., $p \vee \neg q \vee r$).
- **CNF Formula:** A conjunction of ground clauses, which is the standard input for most SAT solvers.

2.2.2 Solving Methodologies: Algorithms and Optimizations

SAT solvers are software systems designed to find satisfying interpretations for propositional formulas. In general, solving techniques are divided into two main categories: complete and incomplete solvers [Bie+09].

Complete Solvers

Complete solvers are guaranteed to find a solution if one exists because they systematically explore the entire search space. While this sounds like a brute-force approach, modern techniques like Conflict-Driven Clause Learning (CDCL) have developed more sophisticated pruning techniques, such as clause learning, which allows them to avoid re-exploring dead ends in the search. Though usually fast, their runtime can still be exponential for certain 'pathologically' hard problem structures.

- **The DPLL Algorithm (David-Putnam-Logemann-Loveland)** [DLL62]
  - **Logic:** The original recursive, backtracking search algorithm. It works by applying Unit Propagation and Pure Literal Elimination rules until no more deductions can be made, then making a Decision on a variable and backtracking upon conflict.
- **Conflict-Driven Clause Learning (CDCL)** [Bie+09]

  - **Logic:** The foundational algorithm for virtually all modern, high-performance complete solvers. CDCL evolved from DPLL but revolutionized field by introducing: Conflict Analysis to find the root cause of a contradiction, Clause Learning to add new constraints, Non-Chronological Backtracking to jump past irrelevant decisions, and periodic Restarts.

- **Binary Decision Diagrams (BDDs)** [Dre01]

  - **Logic:** Rather than seeking out a solution, this method takes an entire Boolean formula and creates a unique, standart graph representation. If the resulting graph is not "null" then the formula is satisfiable. It is not an algorithm for searching, it is a technique for symbolic manipulation.

Incomplete Solvers

Incomplete solvers use local search and stochastic (randomized) methods, operating on a principle of 'iterative improvement'. Instead of systematically building a solution, they start with a complete, random assignment and repeatedly try to 'fix' the unsatisfied clauses by flipping variables. This strategy makes them extremely fast at finding solutions for certain types of large problems. However, their main drawback is their lack of guarantees: they cannot prove a formula is unsatisfiable, and they can get stuck in local optima.

- **GSAT (Greedy SAT)** [SLM92]

  - **Logic:** The pioneering greedy local search algorithm. It starts with a random assignment and, in each step, flips the variable that leads to the largest increase in the total number of satisfied clauses across the whole formula.

- **WalkSAT** [KS96]

  - **Logic:** The most famous and effective variation of stochastic local search. It first picks an unsatisfied clause. Then, with some probability, it flips a variable randomly within that clause (a random walk); otherwise, it performs a greedy flip similar to GSAT. This randomness helps it escape local optima where GSAT would get stuck.

- **Survey Propagation (SP)** [MPZ02]

  - **Logic:** A highly specialized algorithm inspired by statistical physics. It uses a "message-passing" technique on a graph representing the problem to calculate the probability, or "survey," that each variable should be true or false. It's incredibly powerful but also complex and tailored for very hard, random SAT instances near the phase transition.

But

While these solvers are incredibly powerful for pure logic, their inability to natively handle non-Boolean concepts like arithmetic directly motivates the extension to Satisfiability Modulo Theories (SMT), which will be discussed in the following section.

2.3 Satisfiability Modulo Theories (SMT)

As discussed in the previous section, the power of SAT solvers is constrained by their purely propositional nature. While it is, in theory, possible to encode any problem as a Boolean one, doing so typically becomes infeasible and inefficient for applications that require reasoning about non-Boolean things like numbers, arrays, or other structures[DB11]. **Satisfiability Modulo Theories** (SMT) refers to an approach that is designed to close the gap that exists between the power of SAT and the actual needs of high-level modeling.

2.3.1 Core Concepts and Expressiveness

In principle, SMT can be thought of as SAT extended with a specialized "theory solvers" (T-solvers) which are capable of handling additional domains [BT18]. An SMT formula can assert `(x + y > 10)` directly, where `x` and `y` are integer or real variables instead of a simple propositional variable such as `p`. The SMT solver's task is to determine if a satisfying assignment exists for the formula, while respecting the rules of the underlying theory—in this case, the theory of linear arithmetic. This allows for constraints from real-world experiences to be expressed in a natural and compact way.

Common background theories supported by modern SMT solvers include:

- **Linear Arithmetic** over integers and reals (`LIA`/`LRA`).
- **Bit-Vectors** for modeling machine-level arithmetic and fixed-size data.
- **Arrays**, providing a theory for read and write operations on data structures.
- **Uninterpreted Functions**, which allows reasoning about generic functions with the sole property that for the same inputs, the function returns the same output (functional consistency) [BT18].

2.3.2 The Lazy SMT Solving Architecture

The dominant architecture for modern SMT solvers is the "lazy" approach, which cleverly combines a standard SAT solver with one or more T-solvers. The process works as a dialogue between these components [DB11]:

1. **Propositional Abstraction**: First, the SMT formula is simplified by replacing each theory-specific predicate (e.g., `x > 5`) with a fresh Boolean variable (e.g., `p1`). This creates a purely propositional formula that captures the logical structure of the original problem.
2. **SAT Solving**: This abstract formula is passed to the internal SAT solver. If the SAT solver cannot find a satisfying assignment, the original SMT formula is declared unsatisfiable. Otherwise, it proposes a model, which is a set of true-valued propositional variables (e.g., `{p1, ¬p2, p3}`).
3. **Theory Checking**: The theory predicates corresponding to this proposed model are passed to the relevant T-solvers. For instance, if `p1` represented `(x > 5)` and `p2` represented `(x < 3)`, the T-solver would check if the set `{(x > 5), ¬(x < 3)}` is consistent according to the rules of arithmetic.

4. **Conflict Resolution**: If the T-solver finds the set to be consistent, a satisfying model for the original formula has been found. If it is inconsistent (as in the example, since `x` cannot be both greater than 5 and not less than 3), the T-solver generates a "conflict clause" explaining why the combination is impossible (e.g., `¬p1 ∨ p2`, meaning `¬(x > 5) ∨ (x < 3)`). This new clause is added to the propositional formula, and the SAT solver is invoked again to find a different model that does not violate this new constraint.

SAT solvers repeat this process until a model that is consistent with the theory has been found (is satisfiable) or solver terminates when no models can be found (is unsatisfiable). This system uses modern SAT solvers for the hard logical parts, but efficient T-solvers for the more specific less generic reasoning. The power of this approach makes SMT a foundational technology for this dissertation's methodology.

2.4 Bounded Model Checking (BMC)

After introducing the ability of SMT solvers to reason about complex logical formulas, it is time to focus on the technique to generate such formulas: **Bounded Model Checking (BMC)**. BMC, which stands for bounded model checking, was originally proposed as an efficient approach to formal verification, especially for hardware designs, to address the state explosion problem of model checking. BMC does not attempt to prove a property for all possible future states (an unbounded and often undecidable problem). Instead, BMC asks a more constrained question: Does a bug exist within a finite number of execution steps, a bound denoted by k?[Cla+01].

2.4.1 Formal Methodology

The core methodology of BMC is to "unroll" the state-transition system for k steps and translate this finite behavior into a single, large logical formula. This formula is constructed from three key components:

- A formula $I(s_0)$ that describes the set of valid initial states.
- A transition relation $T(s_i, s_{i+1})$ that encodes the rules for moving from a state s_i to the next state s_{i+1}.
- A property $P(s_i)$ that specifies the condition being checked at each step.

The resulting BMC formula asserts that the system starts in a valid initial state ($I(s_0)$), all transitions up to the bound k are valid, and the property P is violated at some point within these k steps. Formally, to find a bug, the formula is:

$$I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge \bigvee_{i=0}^{k} \neg P(s_i)$$

If a satisfiability solver finds a model for this formula, that model represents a specific sequence of states and actions—a **counterexample**—that demonstrates how the property P can be violated. Because the state variables (e.g., positions, costs, velocities) are often non-Boolean, this formula is naturally an SMT problem, making SMT solvers the ideal engine for BMC [BT18].

2.4.2 From Verification to Planning

The crucial insight for this dissertation is that this bug-finding technique can be repurposed for planning. Instead of searching for a path that violates a safety property (something bad must not happen), we search for a path that satisfies a reachability property ("something good must happen"). The planning goal, g, becomes the property we want to achieve. The query is reframed from “Is there a bug within *k* steps?" to “Is there a plan of length *k* that reaches the goal?"

The BMC formula is thus modified to search for a "witness" trace that satisfies the goal at the final step:

$$I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge g_k$$

Here, g_k asserts that the goal condition holds in state s_k. If an SMT solver finds this formula to be satisfiable, the resulting model is no longer a counterexample but is, in fact, the desired artifact: a concrete, step-by-step **plan** of length *k* that is formally verified to achieve the goal. This transformation of a verification technique into a planning engine is the central methodology of this project.

Chapter 3

Implementation and Development

3.1 System Components and Installation

This project integrates several key software components: a custom-built planner using Python and the Z3 SMT solver, the Fast Downward classical planner for benchmarking, and the Matplotlib library for data analysis. This section outlines the installation procedures for each.

Python, Z3, and Pygame Installation

The core SMT-based planner and its visualization tools are implemented in Python.

Z3 SMT Solver

The planner uses the **Z3 SMT Solver** developed by Microsoft Research. To run the system, the Z3 library must be installed with its Python bindings. Full documentation can be found on the official GitHub repository: https://github.com/Z3Prover/z3.

1. Ensure Python (version 3.8 or later) is installed on your system.
2. Install the Z3 library using `pip`:

```
pip install z3-solver
```

Pygame for Visualization

The project includes a visualization component to render solution paths. This requires the `pygame` library.

To install `pygame`, run:

```
pip install pygame
```

Fast Downward Planner Installation

For the experimental comparison, this project uses the **Fast Downward** planning system, a domain-independent classical planning system based on heuristic search. As it is typically run on a Linux-based operating system, the following instructions assume such

an environment. The official source code and most up-to-date installation instructions are available on the planner's website: https://www.fast-downward.org/. A typical installation from source involves these steps:

1. Ensure the necessary dependencies are installed. This usually includes `git`, `cmake`, a C++ compiler like `g++`, and Python.
2. Clone the official repository:

   ```
   git clone https://github.com/aibasel/downward.git
   ```

3. Navigate into the directory and run the build script:

   ```
   cd downward
   ./build.py
   ```

4. Once built, the planner can be invoked on PDDL files. For example, to run an A* search with the $LM - cut$ heuristic:

   ```
   ./fast-downward.py domain.pddl problem.pddl --search
   "astar(lmcut())"
   ```

5. But in case of this application, There is no need to compile anything manually, as PerformanceAnalyser does it so just add fast-downward.py file in the same folder with PerformanceAnalyser.py

Matplotlib for Performance Analysis

The performance analysis graphs and experimental results presented in Chapter 4 were generated using **Matplotlib**, a comprehensive plotting library for Python. It provides a MATLAB-like interface for creating a wide variety of static and interactive visualizations. To install Matplotlib, run:

```
pip install matplotlib
```

In this project, its main purpose is demonstration of performance analysis result by processing raw output data from planners and creating a table.

3.2 An example to demonstrate the power of SMT: Solving Sudoku with Z3

To demonstrate the fundamental principles of Satisfiability Modulo Theories (SMT) and the practical application of the Z3 solver, a classic and well-known constraint satisfaction problem – Sudoku – would be a great example. The logical structure of Sudoku makes it a great study material for translating a set of rules into formal SMT model. This basic exercise checks current Z3 setup and also demonstrates the declerative nature of SMT before applying these techniques to the more complex domain of bounded model checking.

3.2.1 Problem Formulation

Sudoku is a number-placement puzzle played on a 9×9 grid. The objective is to fill the grid with digits from 1 to 9, without repetition of digits in any row, column and sub-blocks. These constraints can be formally stated as follows:

1. **Cell Constraint**: Every cell in the grid must contain exactly one integer from 1 to 9.
2. **Row Constraint**: Each of the 9 rows must contain each digit from 1 to 9 exactly once.
3. **Column Constraint**: Each of the 9 columns must contain each digit from 1 to 9 exactly once.
4. **Subgrid Constraint**: Each of the 9 non-overlapping 3×3 subgrids must contain each digit from 1 to 9 exactly once.
5. **Initial State**: The solution must preserve the digits provided in the initial puzzle.

3.2.2 Encoding Sudoku Rules in SMT

To solve this problem with Z3, it is required to translate each of the above rules into a set of logical constraints. The first step is to define a representation for the Sudoku grid itself. In that example, I modeled the grid as a 9×9 matrix of integer variables, where each variable represents a cell in the grid. In the Z3 Python API, this is achieved by creating a two-dimensional list of `Int` variables:

```
grid = [[Int(f'pos_{i}_{j}') for j in range(9)] for i in range(9)]
```

Let's consider grid of variables as G. Each cell $G_{i,j}$ for $i, j \in \{0, \ldots, 8\}$ is a symbolic integer variable. The Sudoku rules are then asserted as follows:

Cell Constraint

According to our first rule, cell constraint, value of each cell variable $G_{i,j}$ must be between 1 and 9.

$$\forall i, j \in \{0, \ldots, 8\} : 1 \leq G_{i,j} \leq 9$$

This is implemented in Z3 with the following code:

```
for i in range(9):
    for j in range(9):
        solver.add(And(grid[i][j] >= 1, grid[i][j] <= 9))
```

Row and Column Constraints

Secondly, one of the main rules of sudoku – each row and column should include each number at most once, which can be implemented easily by using the `Distinct` constraint provided by Z3. For each row i and each column j, the collection of its 9 variables must be different from each other.

```
# Row constraints
for i in range(9):
    solver.add(Distinct(grid[i]))
# Column constraints
for j in range(9):
    solver.add(Distinct([grid[i][j] for i in range(9)]))
```

Subgrid Constraint

Similarly, for the 3rd rule – all values 3×3 sub grids must be distinct – again, applying the `Distinct` constraint to each of the nine blocks is enough to ensure. To do that, following code iterates through the grid in 3×3 blocks and collects the 9 cells within each block.

```
for block_row in range(3):
    for block_col in range(3):
        subgrid = [grid[i][j]
                   for i in range(block_row*3, block_row*3 + 3)
                   for j in range(block_col*3, block_col*3 + 3)]
        solver.add(Distinct(subgrid))
```

Initial State Constraint

Finally, the initial puzzle configuration is encoded by adding equality constraints. For every cell (i, j) that is pre-filled with a value v in the puzzle, code adds the constraint $G_{i,j} = v$.

```
if puzzle[i][j] != 0:
    solver.add(grid[i][j] == puzzle[i][j])
```

3.2.3 Solving and Results

After loading all constraints into the solver, `solver.check()` are then inkoved. If the problem is satisfiable (`sat`), the solver has found a valid assignment of values to all cell variables that satisfies every asserted constraint. Which means that user can retrieve this solution using `solver.model()`. For instance, given the following puzzle:

```
Unsolved Puzzle:
-------------------------
| . . . | . . . | 2 . . |
| . 8 . | . . 7 | . 9 . |
| 6 . 2 | . . . | 5 . . |
-------------------------
| . 7 . | . 6 . | . . . |
| . . . | 9 . 1 | . . . |
| . . . | . 2 . | . 4 . |
-------------------------
| . . 5 | . . . | 6 . 3 |
| . 9 . | 4 . . | . 7 . |
| . . 6 | . . . | . . . |
-------------------------
```

The Z3 solver efficiently produced the unique valid solution in approximately 0.6851 seconds on a standard machine.

```
Solved Sudoku:
-------------------------
| 9 5 7 | 6 1 3 | 2 8 4 |
| 4 8 3 | 2 5 7 | 1 9 6 |
| 6 1 2 | 8 9 4 | 5 3 7 |
-------------------------
| 1 7 8 | 3 6 2 | 9 5 4 |
| 5 2 4 | 9 8 1 | 3 6 7 |
| 3 6 9 | 5 2 4 | 8 1 2 |
-------------------------
| 8 4 5 | 7 9 2 | 6 1 3 |
| 2 9 1 | 4 3 6 | 8 7 5 |
| 7 3 6 | 1 8 5 | 4 2 9 |
-------------------------
```

This example successfully demonstrates that a complex logic puzzle can be solved with minimal procedural code by simply declaring the problem's rules as a set of constraints. The techniques that are subsequently applied to AI planning in this dissertation work are based on this declarative approach.

3.3 Implementing Bounded Model Checking: The White-Black Puzzle

The previous example, Sudoku, illustrated how SMT can be applied to a static problem. . However, AI planning is a dynamic challenge that requires a set of actions to reach a goal. Therefore, it necessitates a shift to a technique which is capable of modeling state transitions: Bounded Model Checking (BMC). To implement this methodology, the "White-Black Puzzle" is introduced as an transition problem. It is a self-contained dynamic system that requires encoding all core components of BMC—an initial state, transition rules, and a goal property—thereby providing a practical foundation before the approach is scaled to the primary AI planning problem.

3.3.1 Problem Formulation

The White-Black Puzzle is played on a one-dimensional grid of four cells. The grid contains one black tile ('B'), two white tiles ('W'), and one empty space ('E'). The system evolves through a series of discrete steps, where tiles can move or jump into the empty cell. The components of the puzzle's transition system are defined as follows:

- **States**: A state is a configuration of the four cells, e.g., `['B', 'W', 'W', 'E']`.
- **Initial State**: The puzzle begins in the specific configuration `['B', 'W', 'W', 'E']`.
- **Transition Relation**: A move is valid if it fits into one of two rules:

1. A tile may move into an adjacent empty cell. This action has a cost of 1.
2. A tile may jump over exactly one other tile into an empty cell. This action has a cost of 2.

At each step, exactly one action, move or jump, must occur.

- **Goal State**: The goal is to reach any configuration where the black tile is positioned to the right of both white tiles.

3.3.2 BMC Encoding for the Puzzle

Application of the BMC technique involves modeling the puzzle's state transitions over a fixed number of steps, k. The resulting finite-step behavior is subsequently encoded as a single, monolithic SMT formula.

State and Cost Representation

A sequence of states $s_0, s_1, \ldots, s_k$ are defined for a bound k. Each state s_t is a list of four symbolic string variables representing the contents of the grid cells at step t. In addition to that, an integer variable, c_t, is implemented to track the cumulative cost at each step.

```
state = [[String(f'state_{t}_{i}') for i in range(4)]
         for t in range(maxSteps + 1)]
cost = [Int(f'cost_{t}') for t in range(maxSteps + 1)]
```

Initial State Constraint

The formula begins by asserting the initial configuration at step 0 (s_0) and setting the initial cost to zero.

$$(s_{0,0} = \text{'B'}) \wedge (s_{0,1} = \text{'W'}) \wedge (s_{0,2} = \text{'W'}) \wedge (s_{0,3} = \text{'E'}) \wedge (c_0 = 0)$$

This is implemented in Z3 as:

```
for i in range(4):
    solver.add(state[0][i] == inputState[i])
solver.add(cost[0] == 0)
```

Transition Relation

The transition relation $T(s_t, s_{t+1})$ is the most complex part of the encoding. For each step t from 0 to $k-1$, it must be ensured that the state s_{t+1} is a valid successor of s_t. This is done by defining all possible valid moves and jumps from any cell i to any other cell j. The entire transition relation is represented as a large disjunction (an `Or` in Z3), where each clause corresponds to a single valid action.

For example, a "move right" from cell i to $i+1$ at step t is a conjunction of several conditions:

1. **Precondition**: Cell i must contain a tile and cell $i+1$ must be empty in state s_t.
2. **Effect**: In state s_{t+1}, the tile from $s_{t,i}$ moves to $s_{t+1,i+1}$, and cell $s_{t+1,i}$ becomes empty.

3. **Frame Axiom**: All other cells j (where $j \neq i$ and $j \neq i+1$) must remain unchanged between s_t and s_{t+1}. This is critical to prevent the solver from making arbitrary, unrelated changes.

4. **Cost Update**: The cost c_{t+1} is incremented by 1.

A snippet for this specific move is encoded as:

```
And(
    state[t][i] != 'E', state[t][i + 1] == 'E',
    state[t + 1][i + 1] == state[t][i],
    state[t + 1][i] == 'E',
    *[state[t + 1][j] == state[t][j] for j in range(4)
    if j not in [i, i+1]],
    cost[t + 1] == cost[t] + 1
)
```

The full transition relation is the disjunction of all such valid move and jump possibilities for that time step.

Goal Constraint

Finally, the encoding requires that the goal condition is satisfied at the final time step k. The condition that the black tile's index is greater than both white tiles' indices is encoded using implication. For every pair of cells (i, j), assert: if cell i contains 'B' and cell j contains 'W', then it must be that $i > j$.

$$\forall i, j \in \{0, 1, 2, 3\}, i \neq j : (s_{k,i} = \text{'B'} \wedge s_{k,j} = \text{'W'}) \implies i > j$$

Following code implemented the logic by iterating through all pairs of indices:

```
constraints = []
for i in range(4):
    for j in range(4):
        if i != j:
            constraints.append(
                Implies(
                  And(state[maxSteps][i] == 'B',
                    state[maxSteps][j] == 'W'),
                    i > j
                )
            )
solver.add(And(constraints))
```

3.3.3 Results

As the complete formula constructed, it is time to ask Z3 solver to find a model. For a bound of $k = 5$, the solver finds the problem satisfiable and returns a concrete path that reaches the goal. The resulting trace is:

```
Solution found:
Step 0: ['B', 'W', 'W', 'E'] | Cost: 0
Step 1: ['B', 'E', 'W', 'W'] | Cost: 2
Step 2: ['B', 'W', 'E', 'W'] | Cost: 3
Step 3: ['E', 'W', 'B', 'W'] | Cost: 5
Step 4: ['W', 'E', 'B', 'W'] | Cost: 6
Step 5: ['W', 'W', 'B', 'E'] | Cost: 8
Total cost: 8
```

This successful application demonstrates how the abstract principles of BMC can be used to create a concrete solver for a specific state-space search problem which forms the direct foundation for the primary task: encoding the 2D grid planning problem.

3.4 Planning problem on a 2-d grid with obstacles as a BMC problem

3.4.1 The MineField Problem Formulation

To demonstrate power of SMT with BMC, "MineField Problem" was established. The environment is a two-dimensional grid, which contains three types of entities: a single **robot**, one or more pieces of **gold**, and zero or more **obstacles**. The **objective** is to find a sequence of actions for the robot to collect all pieces of gold on the grid. The rules governing the system are as follows:

- The robot can move from its current cell to any adjacent (up, down, left, or right) cell in a single step. Diagonal moves are not permitted.
- The robot cannot move into a cell that is occupied by an obstacle.
- To collect a piece of gold, the robot must be on a cell which contains gold and then execute a 'collect' action while robot still is on that tile.
- The robot can wait on a tile as long as it moved onto that tile.

A solution to the problem is a valid sequence of moves from a given start position that results in a state where all gold pieces have been collected.

3.4.2 System Architecture

The system is implemented as a modular application composed of several components, each responsible for a particular part of the planning and evaluation process. The architecture is organized into three primary areas: the core SMT planner, a framework for performance analysis, and lastly PDDL components required for benchmarking against classical planners.

Core SMT Planner and Utilities

The main components of the project are those that implement the SMT-based planning methodology and provide the user interface.

- **`GoldsInMineField.py`**: The core solver module that implements the BMC encoding.
- **`main.py`**: The primary user interface for running the planner.
- **`DifferentPlanningInstances.py`**: Provides functionalities to create both arbitrary and random planning instances for testing & evaluation of the solver.
- **`visualizer.py`**: The Pygame-based visualization tool for displaying solution paths.

PDDL Integration for Comparative Analysis

To enable the comparative study against classical planners, following components were created:

- **`domain.pddl`**: A static file containing the formal PDDL definition of the MineField domain.
- **`pddl_generator.py`**: A script to generate PDDL problem files from grid instances.

Performance Analysis Framework

A dedicated framework was developed to perform various automated experiments and comparative benchmark with classical planners.

- **`PerformanceAnalyzer.py`**: The main script that automates the experimental process for both the SMT planner and classical planners.

3.4.3 Core Planner: Bounded Model Checking Implementation

The planner is built upon the principles of Bounded Model Checking (BMC). Instead of using a traditional search algorithm, the entire planning problem is translated into a single, large logical formula. This formula is then passed to the Z3 SMT solver. If the solver finds a satisfying assignment (a model), that model directly corresponds to a valid plan. The core function `solve_minefield_plan` in `GoldsInMineField.py` is responsible for constructing this formula. The BMC formula for this planning problem can be expressed as:

$$I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge G(s_k)$$

Where:

- $I(s_0)$ is a set of constraints defining the **initial state** of the system at step 0.
- $T(s_i, s_{i+1})$ is **transition rule**, which encodes the rules of the world that must hold for any step from i to $i + 1$.
- $G(s_k)$ is the **goal condition**, which asserts that the objective has been met at the final step, k.

The following sections detail how each of these components is encoded in Z3.

3.4.4 State Representation and Initial State ($I(s_0)$)

The state of the system at any point in time is the complete configuration of the grid. For a grid of size $R \times C$ and a plan length bound of k steps, the system's evolution is represented by a three-dimensional matrix of Z3 `String` variables: $S_{t,r,c}$ represents the content of the cell at row r, column c, at time step t. To capture the complex interactions between the robot and its environment, a set of possible string values for each cell state are defined:

- `'.'`: An empty cell.
- `'O'`: A static obstacle.
- `'G'`: A cell containing an uncollected piece of gold.
- `'C'`: A cell where gold has been collected.
- `'R'`: The robot on an empty cell.
- `'RG'`: The robot on a cell with uncollected gold.
- `'RC'`: The robot on a cell where gold has been collected.

This state representation simplifies the transition logic by encoding the robot's location and the status of gold within a single variable. The initial state, $I(s_0)$, is encoded by constraining the state variables at $t = 0$ to match the input grid. An additional constraint places the robot at its start position, modifying the cell's state to either 'R' or 'RG' depending on the cell's initial content.

```
# Set initial state based on the input grid
for i in range(num_rows):
    for j in range(num_cols):
        if (i, j) == start_pos: continue
        solver.add(state[0][i][j] == grid[i][j])

# Place the robot at the start position
start_cell_content = grid[start_pos[0]][start_pos[1]]
solver.add(state[0][start_pos[0]][start_pos[1]] ==
           ('RG' if start_cell_content == 'G' else 'R'))
```

3.4.5 The Transition Model ($T(s_i, s_{i+1})$)

The transition logic is the core of the encoding, defining the physics of the grid world. It is implemented in the `add_transition_constraints` function. For every step t from 0 to $k - 1$, a series of constraints are added to ensure that state s_{t+1} is a valid successor of state s_t. A foundational constraint asserts that there must be exactly one robot on the grid at all times. This is encoded using a `Sum` over `If` statements, which count the number of cells containing a robot.

```
robot_indicators = [is_robot_cell(state[t][r][c])
    for r,c in all_cells]
solver.add(Sum([If(indicator, 1, 0)
    for indicator in robot_indicators]) == 1)
```

Robot Movement: The rules for movement are encoded by asserting that if the robot is at $(r_{\text{prev}}, c_{\text{prev}})$ at step t, it must be at a valid successor location $(r_{\text{next}}, c_{\text{next}})$ at step $t+1$. A valid successor is an adjacent (non-diagonal) cell or the same cell, the 'destination' is not an obstacle.

Frame Axioms: A critical part of any BMC encoding is to explicitly state what does *not* change between steps. These "frame axioms" prevent the solver from finding invalid solutions where parts of the world change arbitrarily. The key axioms are:

- Obstacles are static: $\forall t, r, c : (S_{t,r,c} = \text{'O'}) \implies (S_{t+1,r,c} = \text{'O'})$.
- Unaffected cells persist: If a cell is not occupied by the robot at step t or $t+1$, its state remains the same.
- When the robot leaves a cell, the cell reverts to its underlying type. For example, if the robot leaves a cell with gold ('RG'), the cell becomes 'G'. This is crucial for ensuring the world state is consistent.

Action Preconditions: The 'Collect' Action In this model, the 'collect' action is implicit. It occurs when the robot is on a cell with gold ('RG') at step t and remains on that same cell at step $t+1$, changing its state to 'RC'. The function `add_collect_action_constraints` adds the necessary precondition to ensure this action is only valid if the robot was indeed on a gold cell to begin with.

```
# If a collect action occurred, the precondition must have been met
collect_occurred = And(is_robot_cell(state[t][r][c]),
                       state[t+1][r][c] == 'RC')
precondition = state[t][r][c] == 'RG'
solver.add(Implies(collect_occurred, precondition))
```

3.4.6 Goal Formulation ($G(s_k)$)

The final component of the formula is the goal condition, implemented in `add_goal_constraints`. This asserts that the objective must be met at the final step, k. For every cell (r, c) that initially contained gold, its state at step k must be either 'C' (collected) or 'RC' (robot on a collected-gold cell). If the initial grid contains no gold, no goal constraint is added, and any valid sequence of moves is considered a solution.

```
gold_locations = [(r, c) for ... if grid[r][c] == 'G']
if not gold_locations:
    return # No goal to achieve

goal_met_at_step_k = And([
    Or(state[k][r][c] == 'C', state[k][r][c] == 'RC')
    for r, c in gold_locations
])
solver.add(goal_met_at_step_k)
```

By combining these three components—the initial state, the transition rule for all steps, and the final goal—the entire planning problem is translated into a single, comprehensive SMT formula. The successful implementation of this BMC encoding provides the foundation upon which all subsequent analysis is built.

3.4.7 Testing and Verification

Before conducting a large-scale performance analysis on such a project, it is important to check the **correctness** of the SMT encoding and the planning system overall. If the solver produces incorrect results or fails to find a solution for a valid problem, then it is useless to be fast. Therefore, testing and validation strategy is designed which consists of a set of manually designed test cases together with a generator of random instances which are able to produce both purely random and solution-guaranteed cases.

Testing Strategy

The verification process was designed to incrementally test the solver's logic. The principal objectives were to ensure that:

1. The solver correctly distinguishes between solvable and unsolvable planning problems.
2. The transition model, including the frame axioms, prevents illegal state changes and preserves static obstacles.
3. Goal conditions and *cost constraints* (when present) are correctly evaluated.
4. The solver can handle a wide variety of grid configurations and step bounds.

These aims were addressed by two complementary methods implemented in `DifferentPlanningInstances.py`: a suite of predefined "arbitrary" tests for systematic verification and a randomized instance generator that exercises the solver on a wide range of grids.

Arbitrary Test cases for Correctness

A set of ten test cases was created to exercise the core logic. Each test specifies a grid layout, a start position, a maximum number of steps, an expected outcome, and—when relevant—a cost model and maximum total cost. The `run_arbitrary_tests` function executes each case, measures the solve time and reports whether the solver's outcome matches the expected result. Some notable test cases:

Test 1: Impossible Path (Gold Blocked). A single piece of gold is completely surrounded by obstacles. The solver must recognise that the planning problem is unsatisfiable (`unsat`), confirming that the transition model correctly prevents illegal moves.

Test 5: Maximum Steps Too Low. A simple grid contains a reachable piece of gold, but the bound k on the number of steps is too small. The solver returns `unsat`, demonstrating that the bounded model checking framework correctly enforces the step limit and does not fabricate shorter plans.

Test 7: Invalid Start Position. The robot's initial cell is an obstacle. This test checks that the solver validates input before constructing the SMT encoding and promptly reports unsatisfiable problems when the start is illegal.

Test 8: Full Map of Gold. All cells contain gold. This extreme case confirms that the solver can collect multiple pieces of gold efficiently and that the goal is satisfied only once all cells are collected or occupied by the robot.

Tests 9–10: Cost Constraints. Two tests introduce a simple cost model (`'move'`, `'collect'` and `'stay'`) and a maximum total cost. Test 9 assigns each move and collect a cost of 1 and allows three units of cost, which is sufficient to move once and collect. Test 10 raises the per-action costs to 2 while keeping the budget at 3 so it makes problem unsatisfiable. These tests verify that the cost-constraint extension of the SMT encoding is properly integrated into the goal condition and that the solver distinguishes feasible from infeasible cost budgets.

Taken together, the ten arbitrary tests provide a quick, repeatable check of the solver's core logic, initialization, action preconditions and goal evaluation.

Randomized Instance Generation

A fixed test set cannot cover the vast space of possible configurations. To tackle this issue and also to generate problems for performance experiments (Chapter 4), a framework was implemented for generating random instances. The `run_random_instance_mode` function acts as the main entry point for randomized testing. It first parses the user-supplied parameters, calls `generate_reachable_instance` or `generate_random_instance` — if reachability is not required — and then invokes `solve_minefield_plan` on the resulting grid. The solver's output is displayed along with statistics such as execution time and, if asked, the accumulated cost (for details see section 3.4.8) . This workflow allows a large number of randomized experiments to be executed in a uniform manner.

The function `generate_random_instance` constructs a grid of a specified size, populates it with a chosen number of obstacles and gold pieces at random, and selects a random start position on a non-obstacle cell. Parameters such as the number of rows, columns, obstacles, gold pieces, the random seed, and cost constraints can be specified when invoking `run_random_instance_mode`, providing flexibility for experiments.

To ensure that the generated problems are not unsolvable due to unreachable gold, the wrapper function `generate_reachable_instance` repeatedly calls the basic random generator until it finds a grid where all gold cells are reachable from the start. Reachability is checked using a Breadth-First Search (BFS) over the grid. The helper functions `_neighbors` and `_bfs_reachable` implement this search. The `_bfs_reachable` returns all coordinates reachable from the start while avoiding obstacles:

```
def _bfs_reachable(grid, start):
    """Return the set of coordinates reachable from start
    (avoiding 'O')."""
    rows, cols = len(grid), len(grid[0])
    if grid[start[0]][start[1]] == 'O':
        return set()
    q = deque([start])
    seen = {start}
    while q:
        r, c = q.popleft()
```

```
        for nr, nc in _neighbors(r, c, rows, cols):
            if (nr, nc) not in seen and grid[nr][nc] != 'O':
                seen.add((nr, nc))
                q.append((nr, nc))
    return seen
```

Here, `_neighbors` generates the four adjacent cells (up, down, left, right) within the grid boundaries. The BFS ensures that only grids where the set of gold coordinates is a subset of the reachable set are accepted. If all gold cells are reachable, the instance is returned; otherwise the generator tries again up to a fixed number of attempts.

The random instance generator also provides seed control for reproducibility. When a random seed is specified, the same sequence of grids is generated on repeated runs, enabling fair comparisons of solver performance and allowing experiments to be replicated exactly. When no seed is provided, each run produces a fresh set of instances.

Benefits of Randomized Generation.

1. **Robustness Testing.** Generating thousands of distinct, guaranteed-solvable instances helps uncover edge cases or logical errors that may not appear in the hand-crafted test suite.

2. **Performance Evaluation.** Random instances underpin the experimental analysis of Chapter 4, where solver run-time is measured across varying grid sizes, obstacle densities, gold counts, and cost constraints.

By combining a targeted suite of arbitrary tests with a randomized instance generator that filters out unreachable problems, the system achieves both thorough correctness verification and broad coverage. These foundations enable the subsequent performance studies and comparisons with classical planners.

3.4.8 Extensions

To improve the capabilities and usability of core planner, two primary extensions were developed: a system for handling plans with cost constraints and a graphical tool for visualizing the solution paths.

Planning with Cost Constraints

The original planning system focused only on reachability—finding any valid plan to collect all the gold within a given number of steps. A significant extension was implemented to work with the concept of cost, allowing the system to find plans that not only achieve goal but also follow a specified budget. This functionality is designed to be optional for flexibility so the cost-related constraints are only added to the SMT model if a maximum cost is provided by the user.

When a cost budget is specified, the SMT model is enlarged with a sequence of Z3 integer variables, $c_0, c_1, \ldots, c_k$, to represent the total accumulated cost at each step. The logic is implemented in the `add_cost_constraints` function.

Cost Accumulation: The cost model is defined by assigning integer costs to three fundamental action types: 'move', 'collect', and 'stay'. The initial cost, c_0, is always constrained to be zero. For each subsequent step t, the cost c_{t+1} is calculated based on the action inferred from the change between state s_t and s_{t+1}:

- A **'move'** action is inferred if the robot's position is different between step t and $t + 1$.
- A **'collect'** action is inferred if the robot remains in the same cell, but the cell's state changes from 'RG' (Robot on Gold) to 'RC' (Robot on Collected).
- A **'stay'** action is inferred if the robot's position does not change and a 'collect' action did not occur.

For each of these cases, a conditional constraint of the form `Implies(action_occurred, cost[t+1] == cost[t] + action_cost)` is added to the solver.

Goal Formulation with Costs: The goal formulation changes significantly depending on whether a cost constraint is active.

- **Without Cost Constraints:** If no `max_cost` is given, the system defaults to the standard reachability goal. It asserts that all gold must be collected by the final step, k.
- **With Cost Constraints:** If a budget is provided, the goal is modified. The solver now searches for *any* step t (where $1 \leq t \leq k$) at which the goal state is reached *and* the accumulated cost c_t is within the specified maximum budget. This is expressed as a large disjunction:

$$\bigvee_{t=1}^{k} (\text{Goal}(s_t) \wedge (c_t \leq \text{max\_cost}))$$

A snippet of this logic from the `add_goal_constraints` function is as follows:

```
if max_cost is not None and total_cost is not None:
    # Goal with cost constraint
    possible_solutions = [
        And(goal_met_at_step_t[t], total_cost[t] <= max_cost)
        for t in range(1, max_steps + 1)
    ]
    solver.add(Or(possible_solutions))
else:
    # Standard reachability goal
    solver.add(goal_met_at_step_t[max_steps])
```

This extension transforms the planner from a simple reachability tool into a more practical system which is capable of basic optimization.

Path Visualisation

While the SMT solver provides a correct plan as a sequence of logical states, interpreting this raw data can be difficult. To tackle this issue, a graphical visualization tool was developed using the **Pygame** library, implemented in the `visualizer.py` module. The main function, `visualize_solution`, takes the solution path extracted from the Z3 model and creates an interactive, animated display. The user can navigate through the plan step-by-step using the left and right arrow keys.

The interface consists of two main components:

- **Grid View:** A graphical representation of the minefield, where each cell type (empty, obstacle, gold, robot) is rendered with a distinct color and shape. The robot's movement is animated as the user navigates through the steps.
- **UI Panel:** A panel located below the grid displays critical information for the current step, including the step number and the cumulative cost, providing immediate feedback on the plan's execution.

This visualization tool is good-to-have not only for demonstrating the planner's final output but also as a crucial debugging item during development. It made it significantly easier to identify and correct hidden errors in the complex transition and frame axiom constraints. For an example of the interface at the beginning and end of a solution path, see Figures A.1 and A.2 in Appendix A

3.4.9 Implementation of Performance Analyzer System

To properly measure the SMT-based planner's performance and identify its limitations, a custom and flexible evaluation framework was built. This automated system serves as a central hub which not only stress-tests the SMT solver against various parameters but also compares its performance against classical planners using PDDL. The framework is designed to be modular, allowing for the straightforward addition of new analysis types or the integration of other planners in the future.

First, it conducts stress tests on the SMT solver by programmatically generating a wide range of planning scenarios and systematically recording execution time and success rates. This is crucial for understanding how the planner scales with factors like grid size, obstacle density, and goal complexity. Second, to provide a clear benchmark against established techniques, the system integrates with classical planning tools using the **Planning Domain Definition Language (PDDL)**. The framework can translate its internal grid problems into standard PDDL files, enabling a direct, head-to-head comparison against high-performance classical planners. This dual approach is vital for assessing both the intrinsic capabilities of the SMT method and its performance relative to other established methods.

Performance Analyzer: `PerformanceAnalyzer.py`

The `PerformanceAnalyzer.py` script is the main part of the framework that conducts experiments to determine what affects the performance of the SMT solver. The main system works as follows:

1. It creates a sample grid based on the user's request, targeting a specific variable for testing.

2. It invokes the solver(s) and records the execution time for each run.

3. It displays this data using **Matplotlib** for better visualization of performance trends.

In order to have efficient performance analysis, a time limit was initiated and set to **120 seconds** (2 minutes). For the main part of the project, the analyzer has four main functions, each designed to methodically isolate and test a specific variable's impact on solver performance:

- **`analyze_performance_by_grid_size()`**: Assesses how solver performance scales with the dimensions of the grid. The number of gold pieces and obstacles is kept constant to ensure the primary variable is the size of the state space itself.
- **`analyze_performance_by_max_steps()`**: Investigates the relationship between the plan length (the bound k in BMC) and solve time. It uses a fixed grid and incrementally increases the `max_steps` parameter to identify the "phase transition" point where a problem becomes solvable.
- **`analyze_performance_by_gold_count()`**: Measures how goal complexity affects performance by varying the number of gold pieces on a fixed-size grid.
- **`analyze_performance_by_obstacle_density()`**: Explores how the environment's topology impacts planning difficulty by systematically increasing the number of obstacles.
- **`cost_effect_performance_analyser()`**: Explores how the cost values impacts planning difficulty by systematically solving the same problem in 3 different scenarios– no cost, generous max cost, and rescricted max cost.

Last but not least, another collection of functions was developed to compare the performance of the SMT solver with bounded model checking against classic planners using PDDL. The main function, **`analyze_performance_with_pddl_planner`**, controls this benchmark process. It first generates a suite of random or reachable (based on user's request) instances of progressively increasing size. The function then iterates through each generated instance. For each one, it creates a single temporary problem file using the **`pddl_generator`**. It then proceeds to run each selected planner configuration on this same instance. The execution of the Fast Downward configurations (e.g., A* or LAMA) is managed by a helper function, **`_execute_planner_run`**, which handles the subprocess call and parses the output to extract key metrics like search time and plan length. In case user requests, the SMT solver is also called directly within this main loop. This ensures that every planner is benchmarked on the exact same problem file under identical conditions before the temporary file is removed. Finally, after all instances have been tested, the main function gathers the results from all planners and generates comparative plots showing how solve time and plan length scale with grid size for each approach.

PDDL Implementation: `domain.pddl` and `pddl_generator.py`

To benchmark the SMT planner against other classical planners, an integration with the Planning Domain Definition Language (PDDL) was implemented. This allows the same

abstract planning problem to be solved by different engines for a fair comparison. The integration consists of two parts: a static domain definition and a dynamic problem generator.

1. **`domain.pddl`**: This file contains the formal, static definition of the "minefield" world. It defines the `types` of objects that can exist (`robot`, `gold`, `location`), the `predicates` that describe the world state (e.g., `(at ?r - robot ?l - location)`), and the `actions` the robot can perform (`move`, `collect`). The actions are defined with clear preconditions and effects, specifying the "rules of physics" for the environment.

2. **`pddl_generator.py`**: This script acts as a translator, converting a grid instance from the project's internal Python representation into a PDDL problem file that conforms to the rules defined in `domain.pddl`. The `generate_pddl_problem` function dynamically generates the `(:objects)`, `(:init)`, and `(:goal)` sections of a PDDL file based on a given grid and start position. It identifies all unique objects, asserts the initial state (robot position, gold locations, obstacles, cell adjacencies), and constructs the goal condition, which requires all gold to be collected. This automated generation is essential for running large-scale comparative analyses with external planners like Fast Downward.

Summary

In summary, the implementation of the performance analyzer system provides a comprehensive and robust framework for evaluating the SMT-based planner. The core script, `PerformanceAnalyzer.py`, automates the entire experimental process, from generating diverse test instances to systematically measuring and visualizing the solver's performance against key variables like grid size and goal complexity. This allows for a deep analysis of the SMT approach's intrinsic strengths and weaknesses. Furthermore, the integration with PDDL, through the formal `domain.pddl` file and the `pddl_generator.py` script, extends this framework to allow for direct, fair comparisons against classical planners. Together, these components create the necessary toolchain to rigorously validate the planner's correctness and thoroughly benchmark its performance in a broader context.

Chapter 4

Experimental Results and Analysis

To evaluate the performance and scalability of the proposed SMT-based planning approach, a series of experiments were conducted. Following chapter demonstrate the environment in which these experiments were run, the methodology for generating test instances and measuring performance, and the analysis of resulting data.

4.1 Experimental Setup

All experiments were executed on a consistent hardware and software platform to ensure the comparability of results.

4.1.1 Hardware and Software Environment

The experiments were performed on a laptop with the following specifications:

- **Model:** MSI Cyborg 15 A13V
- **Processor:** 13th Gen Intel® Core™ i7-13620H
- **Graphics:** NVIDIA® GeForce RTX™ 4060 Laptop GPU (8GB GDDR6)
- **Operating System:** Dual boot: Windows 10 Home and Ubuntu 24.04.2 LTS (for performance analysis, ubuntu was used.)

The implementation was developed in Python 3. The core of the planning engine relies on the **Z3 SMT Solver** from Microsoft Research, which is accessed via its Python API. Performance data and graphs were generated using the python's `matplotlib` library.

4.1.2 Methodology for Performance Analysis

The evaluation focuses on measuring the time taken by the Z3 solver to determine the satisfiability of the generated BMC formula. This "solve time" is the primary metric for performance, as it represents the core computational effort of the planning task. The seed for all analysis is "123" where time limit was set to 120 seconds– 2 minutes.

The experiments are designed to isolate the impact of five primary variables on solver performance:

- **Grid Size:** The dimensions of the 2D grid world.

- **Plan Length (k):** The maximum number of steps allowed for a solution, corresponding to the bound in the BMC formulation.
- **Number of Gold Pieces:** The number of goals the agent must achieve.
- **Obstacle Density:** The percentage of the grid occupied by impassable cells.
- **Cost Constraints:** How non-restrictive (generous) and restrictive (tight) cost budgets affect solver performance and problem difficulty.
- **Comparative Analysis:** Benchmarking the SMT planner against classical systems (A*, LAMA).

For each experiment, one variable is systematically varied while the others are held constant, allowing for a clear analysis of its specific impact on the computational complexity of the planning problem. The collected data is then used to generate plots illustrating the relationship between each variable and the solver's performance.

4.2 Performance Analysis

This section presents the results of the experiments designed to evaluate the performance of the SMT-based planner. The primary metric is the solver's execution time, measured in seconds, across various problem configurations. The analysis investigates how changes in grid size, plan length, goal complexity, and obstacle density impact the computational difficulty of the planning task.

4.2.1 Impact of Grid Size

The size of the planning grid is a primary factor determining the problem's complexity, as it directly determines the size of the state space. To methodically evaluate its impact, experiments were conducted using the `analyze_performance_by_grid_size()` function, which incrementally increases the grid's dimensions ($N \times N$) while keeping other parameters constant. The reason behind keeping gold and obstacle count as 1 is to isolate the performance result of grid size.

Hypothesis It is expected that the solver's runtime will grow non-linearly with the increase in grid size.

Setup

- **Grid Size**: Varies from 2×2 until 20×20 or untill the timeout is reached.
- **Plan Length Bound (k)**: Fixed at 20 steps.
- **Instance Configuration**: 1 Gold, 1 Obstacle.

Analysis The baseline analysis was performed without cost constraints, making the task a pure reachability problem. The results, presented in Figure 4.1, confirm the hypothesis. The time required to find a solution grows exponentially as the grid size increases, which is consistent with the quadratically growing number of state variables (N^2) for an $N \times N$ grid. The solver handles smaller grids efficiently, but performance degrades sharply after the 14×14 mark, with the analysis halting after the 16×16 instance exceeded the 120-second time limit. The effect of adding cost constraints is examined in Section 4.2.5.

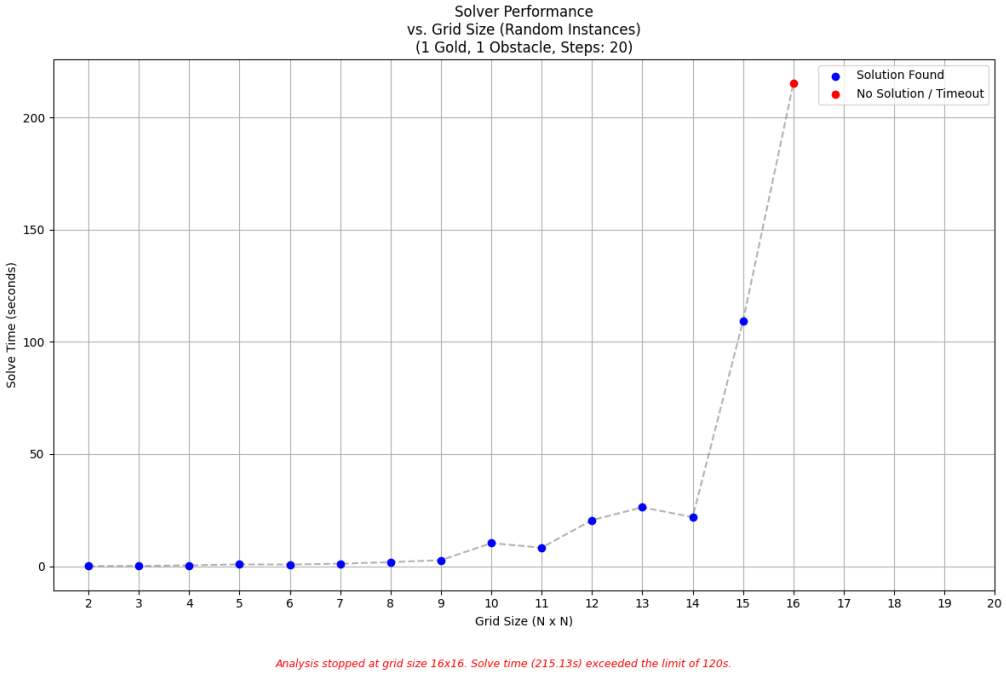

Figure 4.1: Solver performance vs. Grid Size with no cost constraints. The analysis stopped at the 16×16 instance, which exceeded the 120s timeout.

4.2.2 Impact of Plan Length (k)

This experiment analyzes the effect of the bounded model checking depth, k, on performance. A moderately-sized grid (5x5) with 1 gold and 1 obstacle was used, and the maximum number of allowed steps (k) was incrementally increased from 1 to 40.

Hypothesis The solve time is expected to be very low for values of k that are insufficient to reach the goal, as the solver should quickly prove unsatisfiability. At the point where k becomes large enough to admit a solution, the solve time is expected to jump significantly. For all subsequent values of k, the time should generally increase as each additional time step adds more variables and constraints to the SMT formula.

Analysis The experimental results, shown in Figure 4.2, are insightful. For values of k from 1 to 3, the problem is unsatisfiable, and the solver determines this almost instantly (red dots). A clear "phase transition" occurs at $k = 4$, where the goal becomes reachable for the first time and a solution is found. Beyond this point, there is slight linear increase in solve time till k=12, which then the solve time becomes highly volatile and nonstable. While there is a general upward trend, the performance is not linear, instead showing

significant spikes at various points, such as at $k = 29$ and $k = 35$. This behavior suggests that a larger search space (a higher k) does not simply add a fixed amount of work, but can cause the solver to explore computationally difficult parts of the search tree.

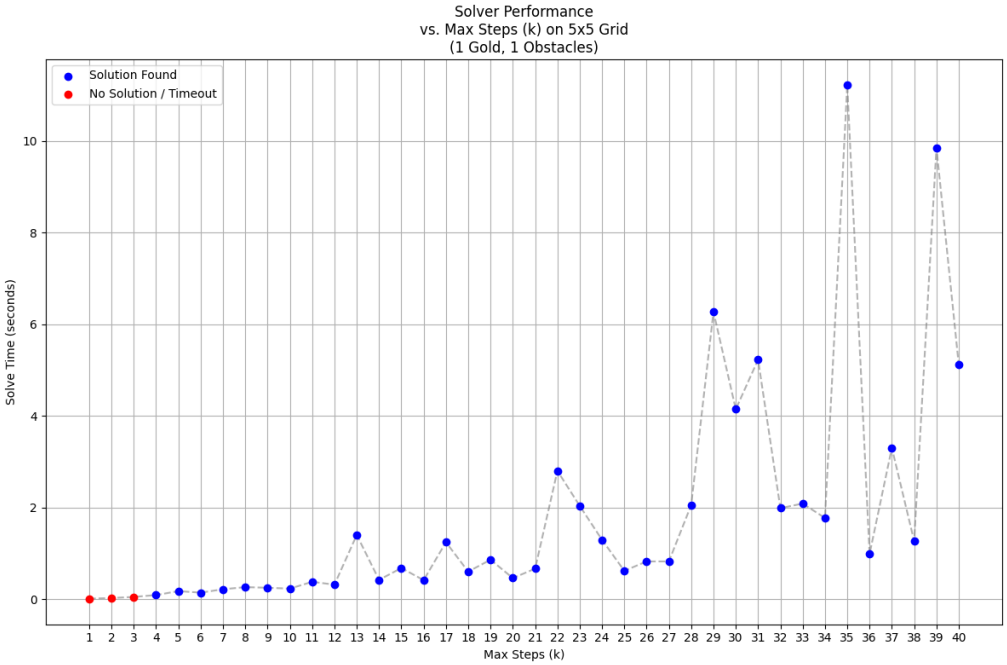

Figure 4.2: Solver performance vs. Max Steps (k) without cost constraints. A clear phase transition from UNSAT (red) to SAT (blue) is visible at k=4.

4.2.3 Impact of Gold Density

This experiment investigates how the complexity of the goal condition affects solver performance. The number of gold pieces serves as a direct measure of goal complexity, as each piece adds a new predicate to the final goal conjunction. To isolate this factor from environmental pathfinding challenges, the analysis was conducted on a fixed-size grid with zero obstacles.

Setup

- **Grid Size**: Fixed at 5×5.
- **Plan Length Bound** (k): Fixed at 30 steps.
- **Obstacles**: Fixed at 0.
- **Number of Gold Pieces**: Varies from 1 up to 15 or until the timeout is reached.

Hypothesis It is expected that increasing the number of gold pieces will lead to a non-linear increase in solve time. A more complex goal requires the solver to satisfy a larger set of simultaneous conditions, which can significantly increase the search effort required to find a valid plan.

Analysis The results, shown in Figure 4.3, confirm that goal complexity is a major driver of computational difficulty. For instances with 1 to 6 gold pieces, the solver finds a solution in under 12 seconds. However, a sharp "tipping point" occurs at 7 gold pieces, where the solve time increases nearly tenfold to over 100 seconds. The problem remains difficult but solvable for 8 and 9 gold pieces. The analysis was terminated when the instance with 10 gold pieces exceeded the 120-second time limit. This demonstrates a highly non-linear relationship between the number of goals and performance, where the problem can rapidly transition from trivial to intractable.

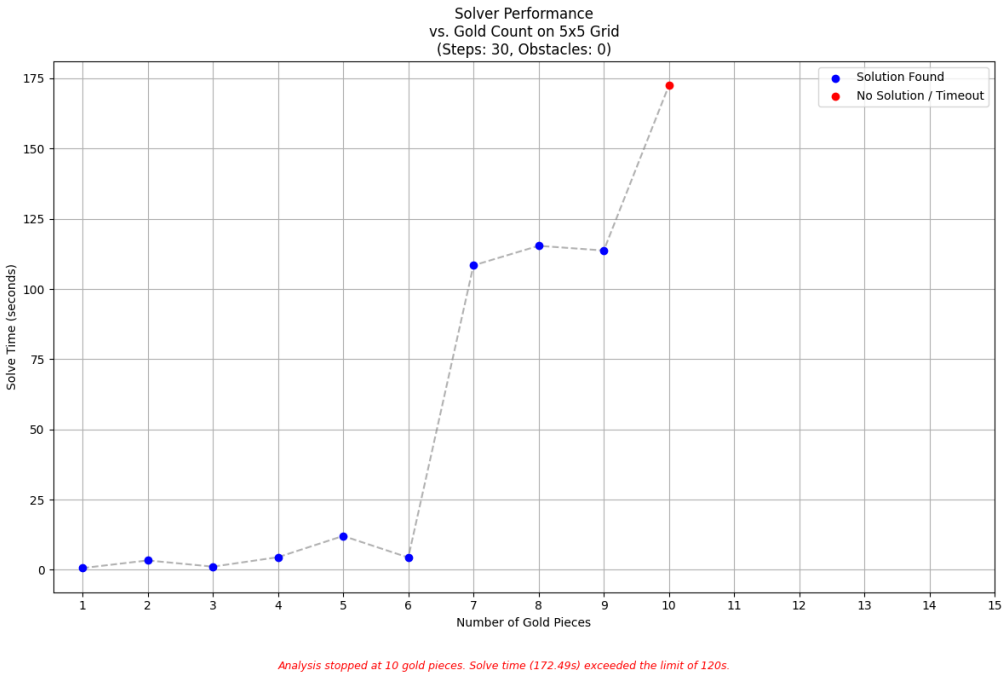

Figure 4.3: Solver performance vs. Number of Gold Pieces on a 5x5 grid. A sharp increase in difficulty is observed at 7 gold pieces, leading to a timeout at 10 pieces.

4.2.4 Impact of Obstacle Density

This experiment explores how the environment's topology impacts planning difficulty by systematically increasing the number of obstacles. Unlike other variables, obstacles can have a dual effect: they can either simplify the problem by pruning the search space or complicate it by creating difficult, maze-like paths. This analysis was conducted on a fixed-size grid with a single gold piece to isolate the effect of environmental constrainedness.

Hypothesis The relationship between obstacle density and solve time is expected to be non-monotonic. It is hypothesized that instances with a low number of obstacles will be solved quickly, as the robot can find a solution directly. Understandably, as the obstacle count increases, the problem should become harder as the solver navigates more complex pathways. However, due to the small grid size (6×6) and the enforcement of reachability, it is expected that at the highest obstacle counts, the problem will become easier again, as the navigable state space will be drastically reduced.

Setup

- **Grid Size**: Fixed at 6×6.
- **Number of Gold Pieces**: Fixed at 1.
- **Plan Length Bound** (k): Fixed at 30 steps.
- **Number of Obstacles**: Varies from 0 up to 30.

Analysis The results in Figure 4.4 confirm the non-monotonic hypothesis, revealing a highly volatile performance profile. Performance improves at very high densities because the reachable state space is drastically reduced. For instance, a 6×6 grid with 30 obstacles leaves at most 6 navigable cells, creating a simple problem for the solver.

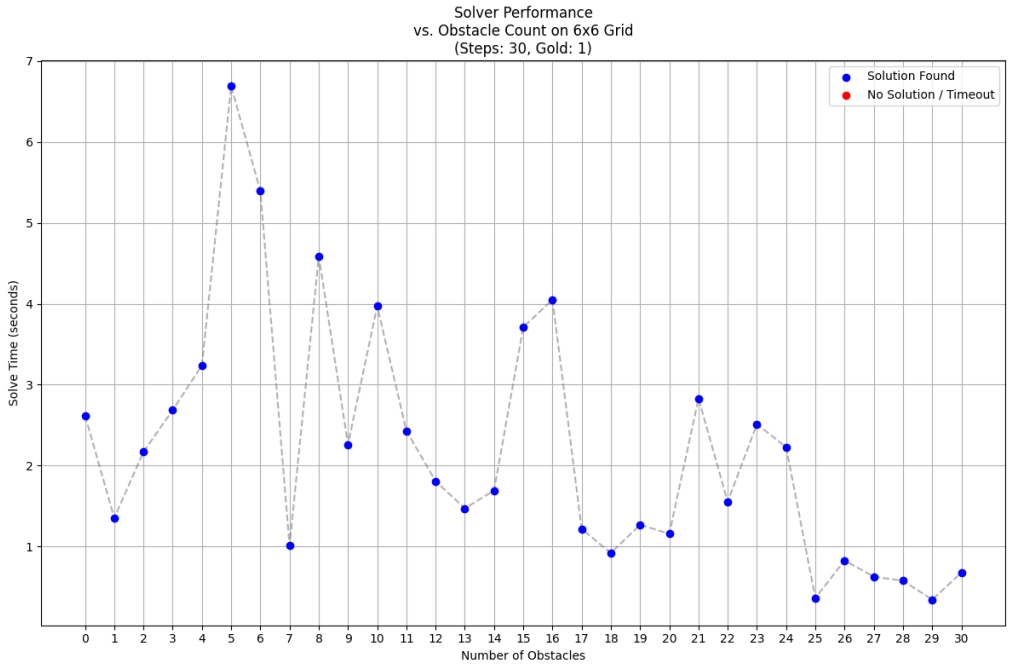

Figure 4.4: Solver performance vs. Number of Obstacles on a 6x6 grid. The relationship is highly non-monotonic and volatile.

4.2.5 Impact of Cost Constraints

This section analyzes how adding arithmetic constraints for plan cost affects solver performance. This capability is a core strength of SMT, allowing the solver to reason over theories such as Linear Integer Arithmetic. The functionality is implemented in the `add_cost_constraints` function, which introduces integer variables to track the accumulated cost at each step. The analysis examines three scenarios across varying grid sizes: no cost, a generous (high-cost) budget, and a restrictive (low-cost) budget.

Hypothesis: It is expected that a non-restrictive cost budget will have a minimal impact on performance, while a restrictive budget will significantly increase the problem's difficulty. A tight constraint forces the solver to navigate a much smaller and more complex solution space, which can make the problem computationally harder, especially near the boundary of satisfiability.

Scenario 1: Common Gold and Obstacle Density

The primary analysis was conducted on instances where both gold and obstacles were generated with "common" density (30% of grid cells).

Setup

- **Grid Size**: Varies from 2×2 up to 10×10.
- **Time limit**: 300 seconds (to obtain more data).
- **Plan Length Bound** (k): Fixed at 100 steps.
- **Density**: Gold and Obstacles set to 'common' (c).
- **Scenarios Tested**:
  1. No Cost
  2. High-Cost Budget – max cost = 150
  3. Low-Cost Budget – max cost = 20

Analysis The results, shown in Figure 4.5, are revealing. The performance of the "No Cost" and "High-Cost Budget" scenarios are nearly identical, confirming that a non-restrictive budget adds minimal overhead. In contrast, the "Low-Cost Budget" scenario shows a completely different profile. It finds a solution for the smallest grid, but fails on the 3×3 instance with a solve time exceeding 400 seconds. This demonstrates that proving a tightly constrained problem is unsatisfiable (or on the edge of satisfiability) is computationally much harder than finding a solution in a less constrained space.

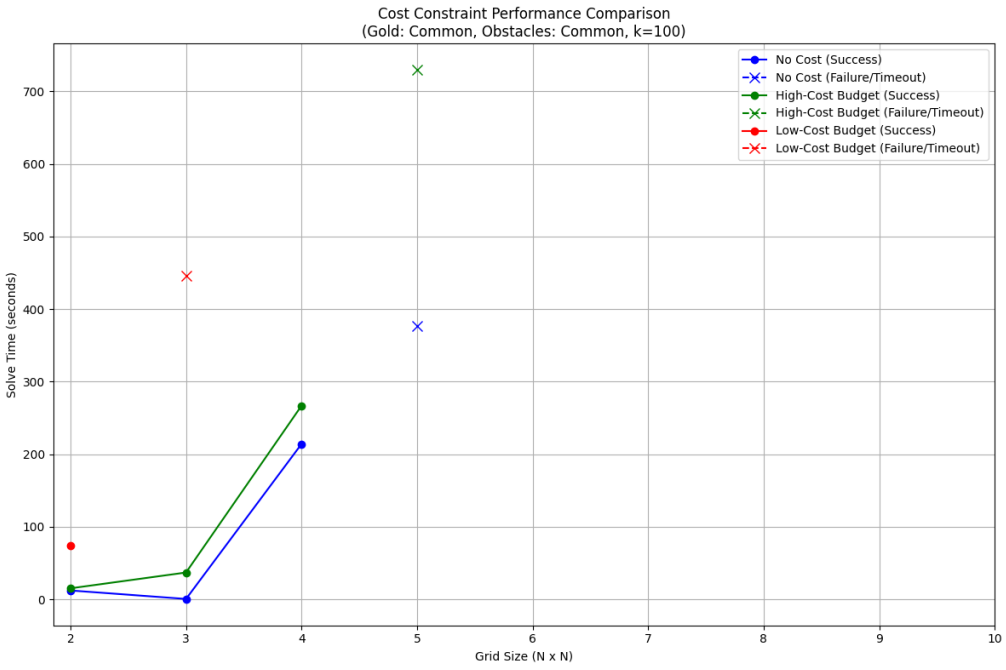

Figure 4.5: Cost constraint performance comparison on instances with common gold and obstacle density.

4.2.6 Comparative Analysis with Classical Planners

To evaluate the performance of the SMT-based approach, a comparative benchmark was conducted against classical planners using the Fast Downward system. The experiment compares the Z3 solver against three classical configurations: A* search with the LMCut heuristic, A* search with the hMax heuristic, and the LAMA satisficing planner.

Hypothesis: It is excepted that the specialized heuristic search algorithms of classical planners will significantly outperform the general-purpose SMT solver in both solve time and plan quality (length). The SMT approach's strength lies in expressiveness, not raw planning speed for classical domains and considering main purpose of SMT approach is checking satifiability, its performance on plan length is also expected to be poor.

Setup

- **Grid Size**: Varies from 2×2 up to 6×6.
- **Density**: Gold and Obstacles set to 'common' (c).
- **Z3 Plan Length Bound (k)**: Dynamically set to 34, based on the optimal plan length for the largest grid plus a small buffer.

Analysis The results, shown in Figure 4.6, clearly support the hypothesis. The top plot shows that the classical planners (A* and LAMA) solve all instances almost instantaneously, with solve times close to zero. In stark contrast, the Z3 solver's runtime, while fast for small grids, grows exponentially and exceeds 200 seconds for the 6×6 instance. The bottom plot, which compares plan length, is equally telling. The classical planners consistently find short, high-quality plans that scale smoothly with grid size. The Z3 solver, however, produces plans of erratic length. Because it is a satisfiability engine, it finds any valid plan within the bound of k=34 and is not optimized to find the shortest one. This results in highly suboptimal plans, especially for larger grids where the plan length hits the maximum allowed bound. This benchmark demonstrates a clear trade-off: while the SMT approach provides great flexibility and formal guarantees, it cannot compete with specialized classical planners on pure speed or plan quality for this type of problem.

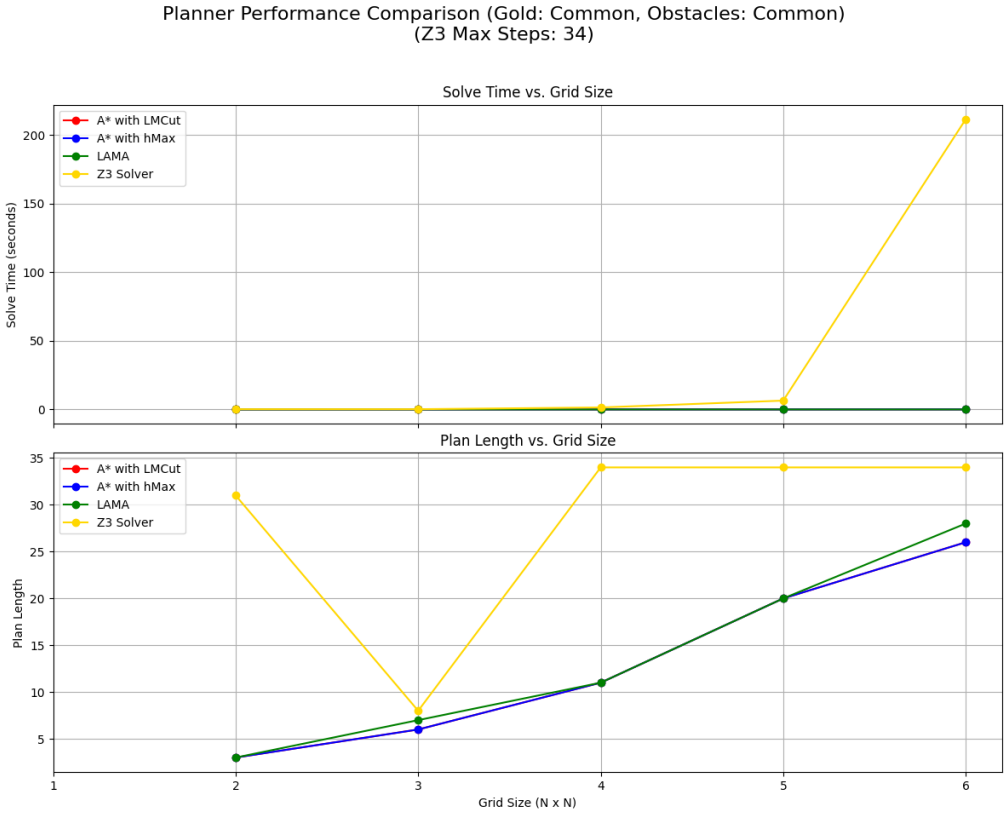

Figure 4.6: Planner performance comparison for solve time and plan length.

4.3 Discussion of Results

The experimental assessment presents a clear-cut and consistent narrative about the nature of SMT-based planning by means of Bounded Model Checking. Although the evidence confirms that the implementation is functionally correct, it also reveals the practical limitations of the methodology. These results show a significant trade-off between expressive power and computational scalability. An examination will then be made of the reasons these performance characteristics appear, and what they imply for SMT as a planning engine.

The Challenge of Combinatorial Complexity

The experiments showed that the planner's performance is very sensitive to the combinatorial complexity of the SMT formula which is unavoidable in the Bounded Model Checking (BMC) encoding. In this approach, the complete planning problem is translated into a single, large logical formula, which gives rise to a variable for the state of each cell at each time step. As a consequence, formula size and complexity increase depending on many factors.

The most direct scaling factor is the **grid size**. As the grid dimensions (N^2) increase, the complexity of the formula grows polynomially. The real problem is not the number of variables, it is arised from the explosion in the number of **transition constraints** that connect those state variables over time. At every time step, for every cell, the solver must think about every possible move and check that the **frame axioms** hold, i.e, what does not

change. This results in an extremely dense network of logical dependencies. In contrast to a heuristic search algorithm, which uses a form of **locality** to explore the state space, the BMC encoding forces the SMT solver to find a globally consistent assignment for the entire space-time volume at once. An absence of a clear, step-by-step search drives the exponential increase in solve time noticed as grid size increases (Figure 4.1).

Similarly, **gold density** also leads to an exponential increase in difficulty (Figure 4.3), but through a different mechanism. Each piece of gold increases size of the final goal conjunction by adding a new predicate. As result, the goal condition becomes more complicated, forcing the solver to satisfy a larger set of simultaneous constraints. As the analysis shows, this creates a sharp "tipping point" where the search effort grows non-linearly, and the problem can rapidly transition from trivial to intractable.

Furthermore, by adding goals, the problem's nature changes fundamentally. It is no longer a simple reachability task of finding a path to a single point. Instead, the solver is implicitly tasked with solving a far more complex **sequencing problem**: it must find a single, continuous path that visits a set of distinct locations in a valid order. The computational difficulty stems from this combinatorial challenge. As the number of goals increases, the number of possible sequences the solver must implicitly consider grows factorially. This explains the sharp "tipping point" observed in the results, where adding just one more goal transforms the problem from trivial to intractable by drastically increasing the complexity of the underlying search for a valid sequence.

The impact of **plan length** (k) is particularly interesting. For insufficient plan lengths, the solver proves unsatisfiability almost instantly. As k increases, the time required to find a solution initially grows, for instance in the range $3 < k < 12$ (Figure 4.4). However, beyond a certain point, the performance is revealed to be non-linear, as the solve time becomes highly unpredictable. This unpredictable performance is primarily due to the **explosive growth of the search space**. Once k exceeds the minimum required length, the solver must navigate a much larger tree of possibilities, which includes many redundant and unnecessarily long paths. This implies that although the formula's size grows linearly with k, the underlying search complexity grows super-linearly. A second factor causing performance spikes is the uneven **solution density**. For small values of k, the density is typically either zero (UNSAT) or one (a single, optimal plan). For larger values of k, the number of valid solutions can vary tremendously; sometimes many solutions exist, while at other times only a few. This unpredictability means the solver can stumble into computationally "easy" or "hard" zones, causing the observed spikes.

Lastly, the analysis of **obstacle density** (Figure 4.4) reveals a highly volatile and non-monotonic relationship between constraint count and solve time. The reason for this unpredictable behavior is the dual role obstacles play, where their impact is defined not by their quantity, but by their **geometric structure**. Strategically placed obstacles can create computationally difficult "choke points" that complicate pathfinding, causing performance spikes even at low densities. Conversely, a very high density of obstacles can severely prune the search space, leaving few valid paths and thus making the problem trivial for the solver. This effect was pronounced in the experiment, as the performance analyzer was designed to always generate solvable instances. Consequently, at high densities, strategically placed obstacles drastically reduced the valid search space, simplifying the problem for the solver.

Analysis of Cost Constraints and Performance

The SMT/BMC approach exhibits a unique strength in its ability to reason over very expressive logical theories; this capability was tested directly in the cost constraint experiments (Figure 4.5). The results of this research are important to understand how the methodology actually operates in practice.

The first observation is that performance is affected drastically. Having a very large budget introduces very little overhead whereas having a very small budget degrades performance severely. When there is a cost restriction, this turns the simplest path search query into a restricted satisfaction problem. For this reason, a restrictive cost restriction becomes very important. The solver is not looking for just any valid plan anymore; the plan is expected to have a fairly strict arithmetic property. This is because proving unsatisfiability requires the solver to search a very large portion of the search space; this is exponentially harder than finding a solution when many solutions exist.

This result shows the expressiveness-complexity trade-off at its core: while SMT provides the power to model and solve intricate constraints, this comes at a steep, and potentially intractable, price.

Comparative Analysis: SMT vs Heuristic Search

The comparative analysis (Figure 4.6) clearly illustrates the difference between a general-purpose tool and specialized one. The overwhelming performance advantage of the Fast Downward planners is not a criticism of Z3, but a testament to the power of **informed, heuristic search**. Classical planners like those in Fast Downward do not view the problem as a single, large logical formula instead, they perform an explicit search through the state-space graph, guided by an intelligently extracted **heuristic** (e.g., $h_{\max}$ or LM-cut) that estimates the distance to the goal. This heuristic provides powerful semantic guidance, allowing the planner to ignore vast, unproductive regions of the search space.

The SMT solver, by contrast, operates at a lower level of abstraction, searching for a satisfying assignment within the logical formula without the benefit of this high-level, domain-specific guidance. This also explains the disparity in **plan quality**. Heuristic search planners are inherently goal-directed. An optimal planner like A* is mathematically guaranteed to find the shortest path. A satisficing planner like LAMA is designed to find high-quality solutions quickly. The SMT/BMC planner, being a satisfiability engine, is designed only to find *any* valid model that fits within the bound k. It has no mechanism or preference for finding shorter or more efficient plans therefore it results in suboptimal plan lengths.

Synthesis and Implications

In synthesizing these findings, a clear picture emerges. The SMT/BMC methodology is a powerful and flexible approach to planning, but it is not a universally superior replacement for classical techniques. Its utility is defined by the problem's characteristics.

For standard planning domains that map well to the PDDL formalism, where the primary challenges are the size of the state space and the efficiency of the plan, heuristic search planners remain the superior tool due to their specialized, guidance-driven algorithms. The SMT/BMC approach, however, is particularly well-suited for a specific class of problems where the core difficulty lies in **complex, non-classical constraints**. If a planning problem requires reasoning about real-valued time, resource consumption, or

other properties that are difficult to model in PDDL, the overhead of the SMT encoding becomes justified. In such scenarios, the ability to declaratively state these complex rules and receive a formally verified plan is an advantage that may outweigh the raw performance limitations. Therefore, in order to choice a proper paradigm, it is important to understand the structure of problem – whether that problem is fundamentally one of navigating a large state-space or one of satisfying a set of intricate, expressive constraints.

Chapter 5

Self-Assessment and Critical Review

This chapter presents an evaluative description of the dissertation project. I will appraise its main accomplishments, recognize its shortcomings, and consider the insights gained throughout the research and development stages. Completing this project has highlighted that the work presented here offers many interesting possibilities for future work.

5.1 Project Achievements

The project accomplished its main purposes, as required in the original proposal. The main aim was to learn about SMT and BMC and apply them to an AI planning problem, which was accomplished fully.

- **Successful Implementation of SMT/BMC Planner:** A complete, functional planner was developed in Python using the Z3 solver. The system successfully translates a 2D grid-world problem into a BMC formula and extracts a valid plan from the resulting model. This demonstrates a solid grasp of the theoretical concepts and their practical application.
- **Development of a Comprehensive Evaluation Framework:** The creation of the `PerformanceAnalyzer.py` script was a significant achievement. This modular tool not only enabled the systematic testing of the SMT planner against key variables but also facilitated a direct, fair comparison with classical planners through its PDDL integration. This framework was essential for generating the empirical evidence that underpins the dissertation's conclusions.
- **Implementation of Key Extensions:** The project successfully went beyond the core requirements to include the "suggested extensions." A system for handling cost constraints using Linear Integer Arithmetic was integrated, showcasing the primary advantage of using an SMT solver over traditional SAT-based methods. Additionally, a Pygame-based visualization tool was developed, which proved invaluable for debugging the complex logical constraints and for presenting the final solution paths.
- **Addition of a Comparative Benchmark:** As an extension to the project, a comparative analysis was performed against high-performance classical planners. Implementing a PDDL integration made direct benchmarking of the SMT planner's relative strengths and weaknesses possible.

5.2 Weaknesses and Challenges

While the project was successful, there were also challenges faced and limitations that it is important to recognize.

- **Performance and Scalability Limitations:** The greatest weakness of the developed planner lies in its performance. The experimental results clearly demonstrate that the BMC encoding leads to a combinatorial explosion, which causes the method to become intractable for larger or more complex planning instances. While this is a key finding of the project, it means the implemented tool is not a practical solution for large-scale planning problems when compared against highly optimized classical planners.
- **Suboptimal Plan Quality:** A direct consequence of the satisfiability-based approach is the lack of plan quality optimization. The planner finds any valid plan within a given step bound, k, but has no means to prefer shorter or more efficient solutions. The comparative evaluation highlighted that the SMT planner often generated long, inefficient paths, unlike the optimal or near-optimal plans from heuristic search planners. An iterative deepening method (incrementally increasing k) could find the shortest plan but would be even more computationally costly.

5.3 Future Work

The findings and limitations of this project open up several interesting directions for future research.

- **Exploring More Expressive Constraints:** The major benefit of the SMT/BMC approach is its expressiveness. Future work could extend the "MineField" problem with complex, non-classical constraints that are difficult to model in PDDL. This could include real-valued time (e.g., actions taking a specific duration), consumable resources (e.g., the robot having a limited fuel supply), or conditional costs (e.g., moving through certain cells costing more fuel). Testing the planner on such problems would better showcase the unique strengths of the SMT paradigm.
- **Integration of Optimization Techniques:** While the current implementation focuses on satisfiability, SMT solvers like Z3 also include optimization capabilities. Future work could explore using Z3's optimization engine to find plans that minimize a certain objective, such as total cost or plan length. This would involve reformulating the goal from a simple satisfiability check to an optimization query, potentially addressing the plan quality weakness of the current implementation.
- **Investigating Alternative Encodings:** This project used a direct step-based BMC encoding. Other, more sophisticated encoding strategies exist that might improve performance. For instance, investigating alternative state representations or more efficient ways to formulate the transition constraints could potentially mitigate the combinatorial explosion to some extent, allowing the planner to handle slightly larger problems.

Chapter 6

How to Use My Project

This chapter provides practical instructions for running the software developed for this dissertation. **Note: The following instructions assume you are working in a Linux-based environment.** The project is organized into several Python scripts, each serving a specific purpose.

6.1 Prerequisites

First, ensure you have installed all the required dependencies as detailed in Section 3.1.

6.2 Running the Project Scripts

There are four primary runnable scripts in this project:

- **`main.py`**: The main entry point for the SMT-based "MineField" planner.
- **`PerformanceAnalyzer.py`**: Used to run the performance analysis and generate the benchmark graphs presented in this dissertation.
- **`sudoku.py`** and **`whiteblackpuzzle.py`**: Example scripts that demonstrate the principles of SMT and Bounded Model Checking on smaller, well-defined problems.

To run any of these scripts, first navigate to the project directory in your terminal, activate your Python environment, and then execute the desired script using the python command.

```
# Example for running the main planner
python main.py
```

6.3 User Scenarios

The two main scripts, `main.py` and `PerformanceAnalyzer.py`, provide interactive command-line menus for different use cases.

6.3.1 Using the SMT Planner (`main.py`)

Running this script starts the main planner and presents a menu with the following options:

- **Solve a Default Instance:** This runs the planner on a predefined "MineField" grid with cost constraints. If a solution is found, you will be prompted to visualize the step-by-step solution path.
- **Solve a Randomly Generated Instance:** This mode allows you to create and solve a custom planning problem. You will be prompted to enter parameters such as grid dimensions, the number of gold pieces and obstacles, and the maximum plan length (k). You can also enable and configure cost constraints. After evaluation, it would ask for grid representation which would show whole solution in graphical 2D grid.
- **Run Arbitrary Test Cases:** This option executes a predefined suite of tests to verify the planner's correctness. The results (`PASS`/`FAIL`) are printed directly to the console.

6.3.2 Using the Performance Analyzer (`PerformanceAnalyzer.py`)

This script is used to replicate the experiments from Chapter 4. When run, it provides a menu with several analysis options:

- **Isolate a Specific Variable:** You can choose to analyze the planner's performance by systematically varying a single parameter, such as grid size, gold count, obstacle density, or plan length, while keeping others constant.
- **Analyze Cost Constraints:** This option tests how different cost budgets (generous vs. restrictive) impact solver performance on the same set of problems.
- **Run Comparative Analysis:** This feature benchmarks the SMT planner against the Fast Downward planner. It automatically generates the necessary PDDL problem files and produces comparison graphs for both solve time and plan length. To run this benchmark, you must place the `fast-downward.sif` file in the same directory as `PerformanceAnalyzer.py`.

After any analysis is complete, the script will display a Matplotlib graph visualizing the results.

Bibliography

[DLL62] Martin Davis, George Logemann, and Donald Loveland. “A Machine Program for Theorem-Proving”. In: *Communications of the ACM* 5.7 (1962), pp. 394–397. DOI: 10.1145/368273.368557.

[Coo71] Stephen A. Cook. “The Complexity of Theorem-Proving Procedures”. In: *Proceedings of the Third Annual ACM Symposium on Theory of Computing*. ACM, 1971, pp. 151–158.

[HTD90] James Hendler, Austin Tate, and Mark Drummond. “AI Planning: Systems and Techniques”. In: *AI Magazine* 11.2 (1990), pp. 61–77.

[SLM92] Bart Selman, Hector Levesque, and David Mitchell. “A New Method for Solving Hard Satisfiability Problems”. In: *Proceedings of the Tenth National Conference on Artificial Intelligence (AAAI-92)*. 1992, pp. 440–446.

[KS96] Henry A. Kautz and Bart Selman. “Pushing the Envelope: Planning, Propositional Logic, and Stochastic Search”. In: *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI’96)*. 1996, pp. 1194–1201.

[McD+98] Drew McDermott et al. *PDDL – The Planning Domain Definition Language*. Tech. rep. CVC TR-98-003/DCS TR-1165. The AIPS-98 Planning Competition Committee. Yale Center for Computational Vision and Control, 1998.

[Wel99] Daniel S. Weld. “Recent Advances in AI Planning”. In: *AI Magazine* 20.2 (1999), pp. 93–123.

[McD00] Drew McDermott. “The 1998 AI Planning Systems Competition”. In: *AI Magazine* 21.2 (2000), pp. 35–55.

[BG01] Blai Bonet and Héctor Geffner. “Planning as heuristic search”. In: *Artificial Intelligence* 129.1-2 (2001), pp. 5–33. DOI: 10.1016/S0004-3702(01)00108-4.

[Cla+01] Edmund Clarke et al. “Bounded Model Checking Using Satisfiability Solving”. In: *Formal Methods in System Design* 19.1 (2001), pp. 7–34.

[Dre01] Detlef Drechsler Rolf Sieling. “Binary decision diagrams in theory and practice”. In: *International Journal on Software Tools for Technology Transfer* 3 (2001), pp. 112–136. DOI: 10.1007/s100090100056.

[KBF02] Sven Koenig, Colin Bauer, and David Furcy. “Heuristic Search-Based Replanning”. In: *Proceedings of the 6th International Conference on Artificial Intelligence Planning and Scheduling (AIPS’02)*. 2002, pp. 294–301.

[MPZ02] Marc Mézard, Giorgio Parisi, and Riccardo Zecchina. “Analytic and Algorithmic Solution of Random Satisfiability Problems”. In: *Science* 297.5582 (2002), pp. 812–815. DOI: 10.1126/science.1073287.

[McC03] T.L. McCluskey. “PDDL: A language with a purpose?” In: *Proceedings of the 13th International Conference on Automated Planning & Scheduling (ICAPS-03)*. Trento, Italy, 2003.

[EH04] Stefan Edelkamp and Jörg Hoffmann. *PDDL2.2: The Language for the Classical Part of the 4th International Planning Competition*. Tech. rep. 195. Freiburg, Germany: Institut für Informatik, Albert-Ludwigs-Universität Freiburg, 2004.

[GL05] Alfonso Gerevini and Derek Long. *Plan Constraints and Preferences in PDDL3: The Language of the Fifth International Planning Competition*. Tech. rep. Brescia, Italy: Department of Electronics for Automation, University of Brescia, 2005.

[Nar+05] Alexander Nareyek et al. “Constraints and AI Planning”. In: *IEEE Intelligent Systems* 20.2 (2005), pp. 62–72. DOI: 10.1109/MIS.2005.27.

[THN05] Sylvie Thiébaux, Jörg Hoffmann, and Bernhard Nebel. “In defense of PDDL axioms”. In: *Artificial Intelligence* 168.1-2 (2005), pp. 38–69. DOI: 10.1016/j.artint.2005.05.004.

[Hel06] Malte Helmert. “The Fast Downward Planning System”. In: *Journal of Artificial Intelligence Research* 26 (2006), pp. 191–246. DOI: 10.1613/jair.1705.

[Bie+09] Armin Biere et al., eds. *Handbook of Satisfiability*. Vol. 185. Frontiers in Artificial Intelligence and Applications. IOS Press, 2009.

[FM09] John Franco and John Martin. “A History of Satisfiability”. In: *Handbook of Satisfiability*. Ed. by Armin Biere et al. Vol. 185. Frontiers in Artificial Intelligence and Applications. IOS Press, 2009, pp. 1–78.

[RH09] Mark Roberts and Adele Howe. “Learning from planner performance”. In: *Artificial Intelligence* 173.5-6 (2009), pp. 536–561. DOI: 10.1016/j.artint.2008.11.009.

[DB11] Leonardo De Moura and Nikolaj Bjørner. “Satisfiability modulo theories: introduction and applications”. In: *Communications of the ACM* 54.9 (2011), pp. 69–77.

[RWH11] Silvia Richter, Matthias Westphal, and Malte Helmert. “LAMA 2008 and 2011”. In: *IPC 2011 Planner Abstracts*. Ed. by Malte Helmert. 2011, pp. 54–57.

[BT18] Clark Barrett and Cesare Tinelli. “Satisfiability Modulo Theories”. In: *Handbook of Model Checking*. Ed. by Edmund M. Clarke et al. Springer, Cham, 2018, pp. 305–343.

[SJJ21] Javier Segovia-Aguas, Sergio Jiménez, and Anders Jonsson. “Generalized Planning as Heuristic Search”. In: *Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling (ICAPS 2021)*. 2021, pp. 569–577.

[AG24] Marco Aiello and Ilche Georgievski. *Introduction to AI Planning*. 2024. arXiv: 2412.11642 [cs.AI].

Appendix A

Path Visualisation Examples

This appendix contains sample outputs from the Pygame-based path visualization tool.

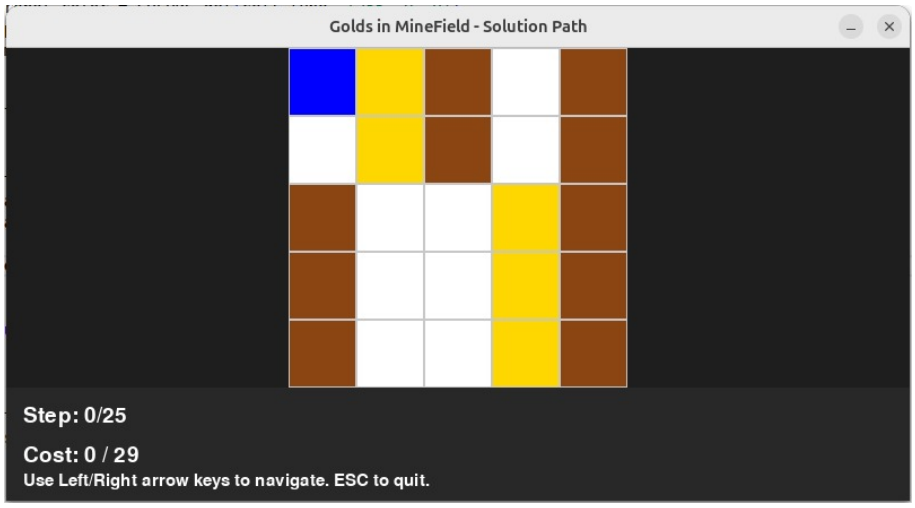

Figure A.1: The visualization tool at the start of a plan (Step 0).

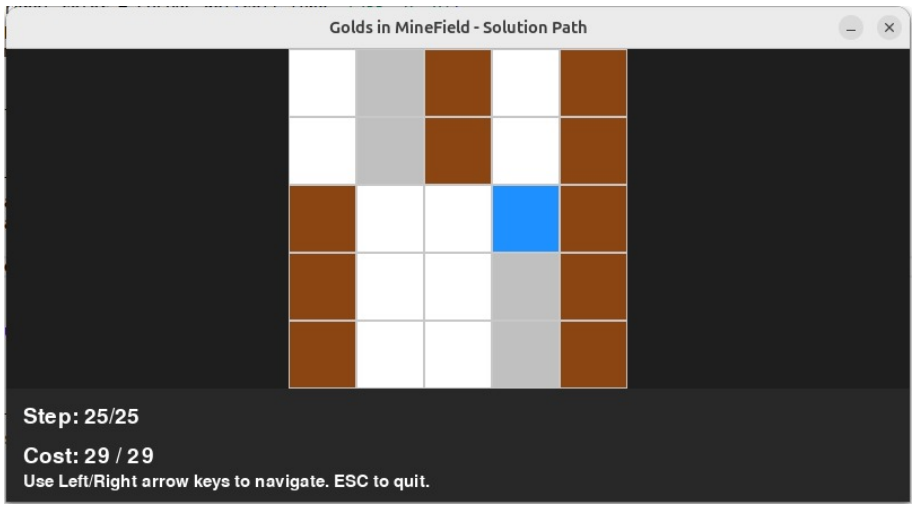

Figure A.2: The visualization tool at the end of a plan (Step 25).